

Article

An LLM-Based Agentic Network Traffic Incident-Report Approach Towards Explainable-AI Network Defense

Chia-Hong Chou ¹, Arjun Sudheer ² and Younghee Park ^{2,*}¹ Department of Computer Science, San Jose State University, San Jose, CA 95192, USA; oliver.chou@sjsu.edu² Department of Computer Engineering, San Jose State University, San Jose, CA 95192, USA; arjun.sudheer@sjsu.edu

* Correspondence: younghee.park@sjsu.edu

Abstract

Traditional intrusion detection systems for IoT networks achieve high classification accuracy but lack interpretability and actionable incident-response capabilities, limiting their operational value in security-critical environments. This paper presents a graph-based multi-agent framework that integrates ensemble machine learning with Large Language Model (LLM)-powered incident report generation via Retrieval-Augmented Generation (RAG). The system employs a three-phase architecture: (1) a lightweight Random Forest binary pre-detection, achieving 99.49% accuracy with a 6 MB model size for edge deployment; (2) ensemble classification combining Multi-Layer Perceptron, Random Forest, and XGBoost with soft voting and SHAP-based feature attribution for explainability; and (3) a ReAct-based summary agent that synthesizes classification results with external threat intelligence from Web search and scholarly databases to generate evidence-grounded incident reports. To address the challenge of evaluating non-deterministic LLM outputs, we introduce custom RAG evaluation metrics—faithfulness and groundedness implemented via the LLM-as-Judge framework. Experimental validation on the ACI IoT Network Dataset 2023 demonstrates ensemble accuracy exceeding 99.8% across 11 attack classes; perfect groundedness scores (1.0), indicating all generated claims derive from the retrieved context; and moderate faithfulness (0.64), reflecting appropriate analytical synthesis. The ensemble approach mitigates individual model weaknesses, improving the UDP Flood F1 score from 48% (MLP alone) to 95% through soft voting. This work bridges the gap between high-accuracy detection and trustworthy, actionable security analysis for automated incident-response systems.

Keywords: agentic system; ensemble learning; explainable AI; intrusion detection; IoT security; large language model; retrieval-augmented generation; SHAP



Academic Editors: Jordi Mongay Batalla, Pascal Lorenz and Ioannis Chatzigiannakis

Received: 25 February 2026

Revised: 3 April 2026

Accepted: 3 April 2026

Published: 7 April 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Advances in Internet of Things (IoT) devices have fundamentally transformed modern industrial infrastructure, healthcare systems, and smart city deployments. However, this rapid expansion introduces critical cybersecurity vulnerabilities, as IoT networks face increasingly sophisticated attack vectors, including Distributed Denial of Service (DDoS) attacks, botnet infiltration, and zero-day exploits. Traditional signature-based Intrusion Detection Systems (IDSs) struggle to adapt to these evolving threats, as they rely on predefined attack patterns and lack the flexibility to detect previously unseen attack variants. Singh et al. emphasize that cloud-based IoT environments require adaptive, context-aware

security frameworks capable of responding to emerging attack methodologies in real time [1]. Furthermore, the black-box nature of many machine learning-based IDS solutions creates a critical trust gap—security analysts cannot verify why an alert was triggered or determine appropriate mitigation strategies without interpretable explanations. This lack of transparency undermines operational confidence and slows incident response, particularly in security-critical environments where false positives and missed detections carry severe consequences.

Existing approaches to network intrusion detection exhibit significant limitations that motivate our research. Machine learning models, while achieving high classification accuracy on benchmark datasets, often operate as opaque decision-makers that provide binary verdicts without actionable explanations [2]. Tariq et al. demonstrated that Explainable AI (XAI) techniques can bridge this interpretability gap by revealing feature attributions and decision boundaries, thereby improving analyst trust and enabling algorithm auditing [3]. Recent work has explored Large Language Models (LLMs) for cybersecurity applications—Kheddar’s survey highlights how Transformer-based architectures enable complex pattern recognition and continuous learning from evolving threat landscapes [4]. Systems such as ChatIDS [5] and HuntGPT [6] leverage LLMs to generate natural-language explanations for IDS alerts, improving comprehensibility for non-expert users. However, these systems primarily explain pre-generated alerts from conventional IDS tools (e.g., Snort and Zeek) rather than producing comprehensive, evidence-grounded incident reports with actionable mitigation strategies derived from external threat intelligence. Additionally, existing LLM-based security systems lack rigorous evaluation frameworks to measure response faithfulness and groundedness—critical metrics for ensuring generated recommendations are based on verifiable evidence rather than plausible-sounding hallucinations. The gap between high-accuracy detection and trustworthy, actionable incident response remains a fundamental challenge in operational security environments.

To address these limitations, we propose a graph-based agentic framework that integrates ensemble machine learning for attack detection with LLM-powered incident-report generation via Retrieval-Augmented Generation (RAG). Our system consists of three sequential phases: (1) a lightweight Random Forest binary classifier with top-10 features performs rapid pre-detection filtering, achieving 99.49% accuracy while maintaining a 6 MB model size for edge deployment; (2) malicious traffic undergoes multi-class classification by an ensemble of three models (Multi-Layer Perceptron, Random Forest, and XGBoost) with soft voting aggregation, followed by SHAP (SHapley Additive exPlanations) analysis [7] to identify the most influential network features; and (3) a ReAct-based [8] summary agent synthesizes classification results, SHAP attributions, and externally retrieved context from Tavily Web search, ArXiv scholarly articles, and an internal knowledge base to generate structured incident reports. The RAG approach grounds LLM outputs in verifiable evidence, reducing the risk of hallucination while enabling the synthesis of threat intelligence from multiple authoritative sources. This agentic workflow enables complex analytical reasoning, connecting attack signatures to observed network features, deriving security implications, and formulating evidence-backed mitigation strategies—capabilities beyond the scope of traditional signature-based or standalone classifier-based IDSs.

The key contributions of this work are listed as follows:

- A novel multi-agent architecture combining efficient pre-detection filtering, ensemble classification with explainability (SHAP), and RAG-based incident-report generation that bridges the gap between high-accuracy detection and actionable, trustworthy security analysis;

- Custom evaluation metrics (faithfulness and groundedness) implemented via the LLM-as-Judge framework to rigorously assess RAG output quality, addressing the challenge of evaluating non-deterministic LLM-generated security recommendations against the retrieved evidence;
- Empirical validation on the 2023 ACI IoT network dataset, demonstrating >99.8% ensemble detection precision in 11 traffic classes; perfect groundedness scores (1.0), indicating that all generated claims derive from the retrieved context; and moderate faithfulness (0.64), reflecting appropriate synthesis rather than mere paraphrasing;
- Evidence that ensemble soft voting provides resilience to individual models' weaknesses, particularly for challenging attack types like UDP Flood, where MLP achieves only a 48% F1 score but ensemble recovers to 95% through XGBoost and Random Forest contributions.

The remainder of this paper is organized as follows: Section 2 reviews related work in machine learning-based IDSs, explainable AI, LLMs for cybersecurity, and RAG systems. Section 3 describes our methodology, including the system architecture, dataset pre-processing, model configurations, and evaluation metrics. Section 4 presents experimental results for binary classification, ensemble performance, per-class F1 analysis, and RAG quality assessment. Section 5 discusses our findings, system limitations, and future research directions.

2. Related Work

2.1. Machine Learning and Deep Learning for Intrusion Detection

Machine learning and deep learning models have been widely adopted for network traffic classification in IoT environments. Verma et al. concluded that boosted classifiers offer the best balance between accuracy and response time among single, ensemble, and boosted models [9]. Deep learning approaches, including RNN-based [10], stacked auto-encoder plus Random Forest [11], and systematically tuned DNN architectures [12], have reported accuracies between 85% and 99% on benchmark datasets such as KDD Cup '99 and NSL-KDD, generally surpassing classical ML baselines in both binary and multi-class settings. However, these approaches share common limitations: weak performance on rare attack classes due to data imbalance, limited generalization beyond benchmark datasets, and a lack of interpretability for operational security teams. This motivates our integration of explainability mechanisms for informed decision-making.

Recent surveys reinforce these findings while highlighting persistent challenges. Abdulganiyu et al. conducted a systematic literature review of network IDSs, identifying that ensemble and hybrid models consistently outperform single classifiers but noting that most studies evaluate models against legacy benchmarks rather than modern IoT traffic [13]. Diana et al. provided an updated overview of IDS techniques for computer networking, emphasizing the shift toward deep learning while cautioning that model complexity often comes at the cost of interpretability and deployment feasibility in resource-constrained environments [14]. Moustafa et al. proposed an ensemble IDS using novel statistical flow features specifically designed for IoT traffic, achieving strong detection rates on the Bot-IoT dataset and underscoring the importance of domain-specific feature engineering for IoT environments [15]. These studies collectively motivate our approach: an ensemble that balances accuracy with interpretability, evaluated on a modern IoT dataset rather than against legacy benchmarks.

2.2. Ensemble Methods for Network Security

Ensemble learning has demonstrated superior performance over single classifiers in network security. Hossain and Islam showed that a Random Forest-based ensemble

achieved over 99% accuracy across multiple public intrusion datasets [16], while Gao et al. reported improved detection using an adaptive weighted voting ensemble combining decision trees, kNN, DNN, and Random Forest on NSL-KDD [17]. Motivated by these findings, we implemented a heterogeneous ensemble combining MLP, Random Forest, and XGBoost to leverage their complementary strengths.

More recently, Alotaibi et al. proposed an ensemble learning framework for enhancing IoT device security, demonstrating that combining multiple classifiers improves the detection of diverse attack types in heterogeneous IoT environments [18]. Almotairi et al. showed that coupling machine learning-based feature selection with ensemble models yields significant accuracy improvements for IoT intrusion detection [19]. Ahmed et al. introduced a hybrid bagging-and-boosting approach with SHAP-based feature selection, bridging the gap between ensemble performance and model interpretability [20]. However, in all these systems, ensemble outputs remain numerical predictions without actionable explanations for security analysts—a gap our framework addresses through SHAP-enhanced natural language reporting.

2.3. Explainable AI (XAI) in Cybersecurity

AI-based intrusion detection systems often lack interpretability [2]. Gaspar et al. applied SHAP and LIME to an MLP-based IoT IDS, revealing how specific features affect classification decisions [21]. Abou El Houda et al. combined RuleFit, LIME, and SHAP to provide local and global explanations for a deep learning IDS on NSL-KDD and UNSW-NB15, improving transparency and expert trust [22]. Our work extends these XAI approaches by combining SHAP-based feature attribution with LLM-generated natural language explanations, bridging the gap between technical feature importance and actionable security insights.

Wang et al. proposed a broader explainable ML framework for IDSs that systematically compares multiple explanation methods, including SHAP, demonstrating that post hoc explanations can maintain model-agnostic interpretability without sacrificing detection performance [23]. Younis et al. specifically applied SHAP to CNN-based intrusion detection, showing that Shapley additive explanations can effectively identify which traffic features drive classification decisions [24]. Ahmed et al. used SHAP not merely for explanation but as a feature selection mechanism in a hybrid bagging–boosting IDS, demonstrating that explainability tools can serve dual purposes [20]. A common limitation across these works is that SHAP outputs remain as numerical attribution scores or visualizations intended for ML practitioners—they are not translated into natural-language explanations accessible to security operations teams, nor are they used as structured input for downstream reporting systems. Our work addresses this gap by feeding SHAP attribution outputs directly into a RAG-based summary agent that generates human-readable incident reports grounded in external threat intelligence.

2.4. LLMs and LLM-Based Intrusion Detection

Large language models have demonstrated strong capabilities in contextual understanding and summarization [25], addressing interpretability limitations in XAI by producing human-readable output [26]. Frameworks such as LangChain implement the ReAct paradigm [8], enabling LLM agents to perform chain-of-thought reasoning and tool use autonomously. Gupta et al. demonstrated that LLMs can automate incident triage, log analysis, and reporting, turning technical telemetry into natural-language summaries [27].

In the IDS domain specifically, Juttner et al.'s ChatIDS uses an LLM to translate conventional IDS alerts into conversational explanations with suggested response actions [5]. Ali et al.'s HuntGPT wraps a Random Forest classifier with SHAP/LIME explanations and

a GPT-3.5-Turbo chatbot to produce readable summaries for security teams [6]. However, both systems remain proof-of-concept prototypes: ChatIDS relies on proprietary LLMs with unresolved trustworthiness concerns, while HuntGPT uses the outdated KDD99 dataset and lacks operational deployment evaluation.

2.5. Retrieval-Augmented Generation for Security

Retrieval-Augmented Generation (RAG) addresses LLM hallucination by grounding responses in retrieved documents, with demonstrated benefits in domains such as public health QA [28] and medical decision support [29]. In cybersecurity, Alhuzali proposed RAGIntel, a RAG-based system that retrieves structured threat intelligence from MITRE ATT&CK for context-aware attack investigation [30].

Borah et al. demonstrated that RAG enables LLMs to adapt to emerging cyber threats by retrieving up-to-date vulnerability and exploit information, reducing hallucination in cybersecurity question answering [31]. Tellache et al. proposed the leveraging of LLMs with cyber threat intelligence for autonomous incident response, showing that grounding LLM reasoning in structured CTI databases improves response accuracy [32]. On the evaluation side, Chen et al. established benchmarks for assessing LLM performance in RAG settings across multiple dimensions, including faithfulness and relevance [33], while Gan et al. provided a comprehensive survey of RAG evaluation methodologies in the era of LLMs [34]. These evaluation frameworks inform our approach to measuring report quality through custom faithfulness and groundedness metrics.

However, existing RAG applications in cybersecurity focus on threat intelligence analysis rather than real-time incident response, and none evaluates output quality with rigorous faithfulness and groundedness metrics as we propose.

2.6. Gap Analysis and Contribution

Table 1 summarizes how representative prior works compare across four dimensions: detection method, explainability, reporting automation, and evaluation rigor. Several gaps emerge from this analysis.

Table 1. Comparison of related approaches with the proposed framework. ✓ = supported; × = not supported; bold row indicates the proposed framework.

Reference	Method	XAI	NL Report	Limitation
Moustafa [15]	Ensemble	×	×	No XAI or reporting
Hossain [16]	RF ensemble	×	×	No explainability
Alotaibi [18]	Ensemble (IoT)	×	×	No XAI
Wang [23]	ML + SHAP	✓	×	No reporting
Gaspar [21]	MLP + SHAP	✓	×	No downstream use
Ahmed [20]	Bag/Boost + SHAP	Partial ^a	×	Selection only
ChatIDS [5]	LLM + alerts	✓	✓	Proprietary, no RAG
HuntGPT [6]	RF + SHAP + GPT	✓	✓	KDD99, no eval
RAGIntel [30]	RAG + MITRE	×	✓	No IDS, no F&G
Ours	Ensemble + SHAP + RAG	✓	✓	Single dataset

^a SHAP used for feature selection, not explanation. NL = Natural Language; F&G = Faithfulness & Groundedness.

First, traditional ML-based IDSs achieve high accuracy but lack explainability, producing numerical outputs that require expert interpretation. Second, existing XAI approaches (SHAP and LIME) for IDSs generate feature-attribution scores or visualizations but do not translate these into actionable natural-language reports. Third, LLM-based IDS tools such as ChatIDS and HuntGPT demonstrate the potential of natural language explanations but

rely on proprietary models, lack retrieval-augmented grounding, and have not been evaluated with quantitative faithfulness or groundedness metrics. Fourth, RAG applications in cybersecurity focus on threat intelligence retrieval rather than incident-response integration.

Our framework addresses these gaps through several novel contributions:

1. **End-to-end pipeline:** We integrate ensemble classification, SHAP-based explainability, and RAG-grounded incident reporting in a single multi-agent architecture, whereas prior works address, at most, two of these three components.
2. **SHAP as structured RAG input:** Unlike prior XAI-for-IDS work, where SHAP outputs serve only as post hoc visualizations, our system feeds SHAP feature attributions as structured context into the RAG retrieval and generation pipeline, enabling the LLM to ground its explanations in both the model's decision rationale and external threat intelligence.
3. **Quantitative RAG evaluation:** We introduce faithfulness and groundedness metrics evaluated via an LLM-as-Judge framework. To the best of our knowledge, no prior IDS + LLM system has applied such rigorous output quality evaluation.
4. **Lightweight two-phase detection:** A compact Random Forest pre-filter (6 MB) enables edge deployment for binary traffic filtering before full ensemble classification, addressing IoT resource constraints.
5. **Graph-based multi-agent orchestration:** The ReAct-based summary agent autonomously performs chain-of-thought reasoning with tool use, unlike the single-prompt approaches of ChatIDS and HuntGPT.

3. Methodology

3.1. System Architecture

Figure 1 presents our graph-based agentic network traffic incident-report system designed for detection accuracy, efficiency, and explainability. The graph-based solution enables conditional paths that drop benign traffic early, avoiding expensive ensemble inference and further RAG generation. The system consists of three main phases followed by evaluation, operating sequentially, and maintains a centralized state shared across phases for communication of processed information.

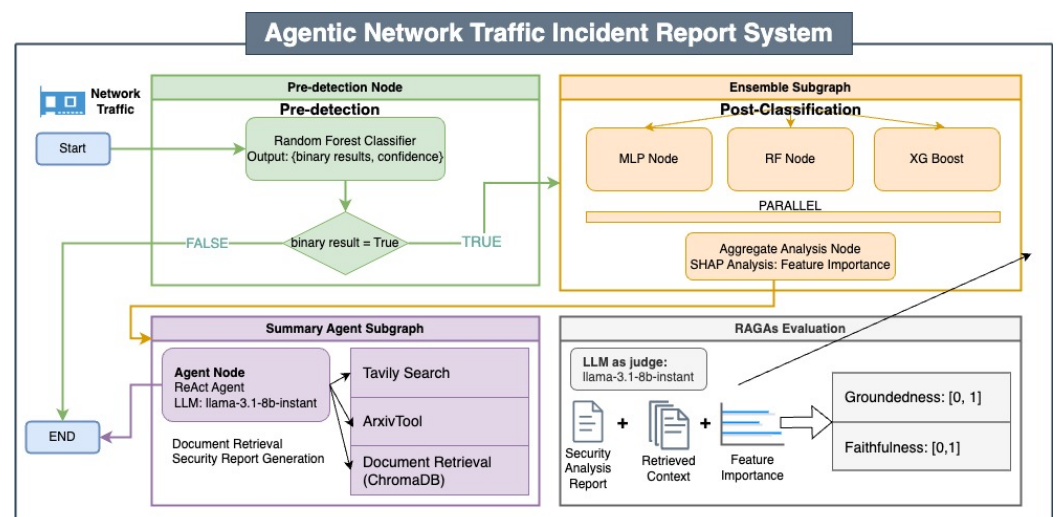


Figure 1. Architecture diagram. Arrows indicate data flow between pipeline phases; colors distinguish the three processing stages (pre-detection, ensemble classification, and summary agent).

3.1.1. Phase 1: Pre-Detection Node

The input consists of single pre-processed network flow sample represented as a 71-dimensional numerical feature vector, normalized using z-score standardization, together with its ground-truth label (used only for evaluation). Only the top-10 features identified by MDI ranking are read by this phase; the remaining 61 features are carried forward in the shared state but not consumed here.

Processing: The reduced feature vector is evaluated by a lightweight binary Random Forest classifier. The model produces a probability distribution over two classes—benign and malicious—and applies a decision threshold to produce a discrete label with an associated confidence score.

The output: consists of a binary classification decision (benign or malicious) and its confidence score written to the shared pipeline state. If the sample is classified as *benign*, the pipeline terminates immediately via the early-exit mechanism, bypassing ensemble inference and report generation entirely. If the sample is classified as *malicious*, it is forwarded to Phase 2, along with the binary result stored in state.

3.1.2. Phase 2: Ensemble Subgraph

The input consists of the full 71-feature normalized network flow sample carried forward from Phase 1, together with the binary classification result and confidence score stored in the shared state.

Processing: The sample is evaluated in parallel by three independently trained multi-class classifiers: Multi-Layer Perceptron (MLP), Random Forest (RF), and XGBoost. Each classifier produces a probability distribution over 10 attack classes (e.g., DNS Flood, Slowloris, and Port Scan). The per-model probability distributions are aggregated through soft voting, where the mean probability is computed across all three models for each class, and the class with the highest mean probability is selected as the consensus prediction. Following classification, SHAP (SHapley Additive exPlanations) [7] analysis is performed on the RF and XGBoost models to compute per-feature contribution scores, quantifying the influence of each network feature on the final prediction.

The output consists of the consensus attack-class label, per-model class probabilities and individual predictions, and a ranked list of SHAP feature-importance values for the predicted class. All outputs are written to the shared pipeline state for downstream consumption by Phase 3.

3.1.3. Phase 3: Summary Agent Subgraph

The input consists of the shared pipeline state accumulated from Phases 1 and 2, comprising (a) the binary classification result and confidence; (b) the ensemble consensus attack label, along with individual model predictions and probability scores; (c) SHAP feature attribution scores ranking the most influential network features; and (d) the raw normalized values of the network features. This structured context is injected into the LLM prompt to enable context-aware, evidence-grounded generation.

Processing: The summary agent, implemented using the ReAct [8] paradigm with the Llama-3.1-8b-instant model, performs iterative retrieval and reasoning. It issues queries to three external tools: Tavily Web Search for real-time threat intelligence, ArXiv Search for academic literature on attack patterns, and a Chroma DB internal knowledge store for historical incidents. The retrieved documents are combined with the shared state context and passed to the LLM, which reasons over the aggregated evidence to construct a structured incident report. This Retrieval-Augmented Generation (RAG) approach ensures the report is based on context rather than LLM text-completion output, which is known to be subject to hallucination and fabrication.

The output is a structured security analysis report comprising the overall classification verdict with confidence and threat level, an attack analysis identifying the specific attack type and threat vector, key features derived from the SHAP analysis, reasoning with inline citations to retrieved evidence, and mitigation recommendations grounded in retrieved threat intelligence. This report is the terminal output of the pipeline.

3.1.4. RAGAs Evaluation

For validation, the incident response output is evaluated using LLM-as-Judge [35] via the RAGAs evaluation framework [36]. The responses are assessed based on two custom-defined metrics driven by prompts: (i) faithfulness and (ii) response groundedness. Further details of the evaluation methods are discussed in Section 3.7.2.

3.2. Dataset

We use the ACI IoT Network Dataset 2023 [37]. The dataset has 1,170,570 records of network flow data categorized into 11 classes. They are: Benign, DNS Flood, ICMP Flood, SYN Flood, UDP Flood, Slowloris, OS Scan, Ping Sweep, Port Scan, Vulnerability Scan, and Dictionary Attack. Each sample consists of 71 numerical features of flow identifiers, temporal statistics, volume statistics, flags, window sizes, and behavioral metrics. The original dataset contains non-numerical identifiers, such as source and destination IP addresses, which are removed during preprocessing.

3.3. Pre-Processing

To prepare our machine learning models, we followed the following steps. (a) Data cleaning: remove invalid NaN and infinite values and drop duplicate entries. (b) Encoding: encode the protocol feature into numerical values and map target labels to integers between 0 and 10, where 0 represents benign traffic. (c) Scaling and normalization: apply a standard scaler with z-score normalization to ensure a zero mean and unit variance for our features. (d) Train–test split: divide the dataset into 80% training and 20% testing sets using stratified sampling to preserve the class distribution across all 11 traffic categories. To prevent data leakage, all preprocessing steps (cleaning, encoding, and scaling) were fit exclusively on the training partition; the learned parameters (e.g., scaler mean and variance) were then applied identically to the held-out test set. Feature selection via mean decrease in impurity (Section 3.4) was likewise computed only from training-set Random Forest models. Hyperparameter tuning for each base learner (Random Forest, MLP, and XGBoost) was conducted via 10-fold stratified cross-validation within the training set, ensuring that no test-set information influenced model configuration. The final soft-voting ensemble was evaluated once on the untouched test partition.

3.4. Pre-Detection

The pre-detection module is the first step a sample goes through. Its goal is to quickly filter out benign traffic so that the subsequent computationally intensive workflow is dedicated only to potential threats. We used a small binary Random Forest classifier for its speed and robustness. To further improve the model's efficiency, the top-10 critical features were identified through Mean Decrease in Impurity (MDI) selection. The selected features are: Packet Length Mean, Fwd Segment Size Avg, Packet Length Max, Subflow Fwd Bytes, Fwd Packet Length Max, Fwd Packet Length Mean, Average Packet Size, Idle Max, Flow IAT Std, and ACK Flag Count. Hyperparameters were tuned using the Optuna framework, including the number of estimators, maximum tree depth, minimum samples required to split a node, and minimum samples per leaf node.

3.5. Ensemble Subgraph

Traffic marked as malicious is then passed to the ensemble subgraph for post classification (Figure 2), which is similar to Hossain & Islam's [16] approach. This module is designed to identify the specific attack type using a collection of three machine learning models: Random Forest (RF), Extreme Gradient Boosting (XGBoost), and Multi-Layer Perceptron (MLP). Each model is trained on the 71-feature set using 10-fold stratified cross-validation and hyperparameter-tuned with Optuna to maximize the weighted F1 score. The model configurations and architectures are exemplified in Appendix A.

The outputs of the three classifier models are aggregated through soft voting to produce a consensus prediction. Unlike hard voting, which selects the class with the majority of discrete label votes, soft voting averages the predicted class probability distributions from each model and selects the class with the highest mean probability. Assiri et al. define majority (hard) voting as the selection of the class with the most votes and "soft voting" as prediction based on the predicted probabilities of each classifier, including an "average of probabilities" rule [38].

Formally, for K models and C classes, the ensemble prediction is

$$\hat{y} = \arg \max_{c \in C} \frac{1}{K} \sum_{k=1}^K P_k(y = c | \mathbf{x}),$$

where $P_k(y = c | \mathbf{x})$ is the probability of class c predicted by model k for input $\mathbf{x}$. This approach preserves confidence information from each classifier—for example, if MLP predicts UDP Flood with 40% confidence while both Random Forest and XGBoost predict it with 95% confidence, the averaged probability still strongly favors the correct class, despite MLP's uncertainty. Equal weights ($1/K$) are used for all three models to avoid overfitting to validation set-specific performance rankings.

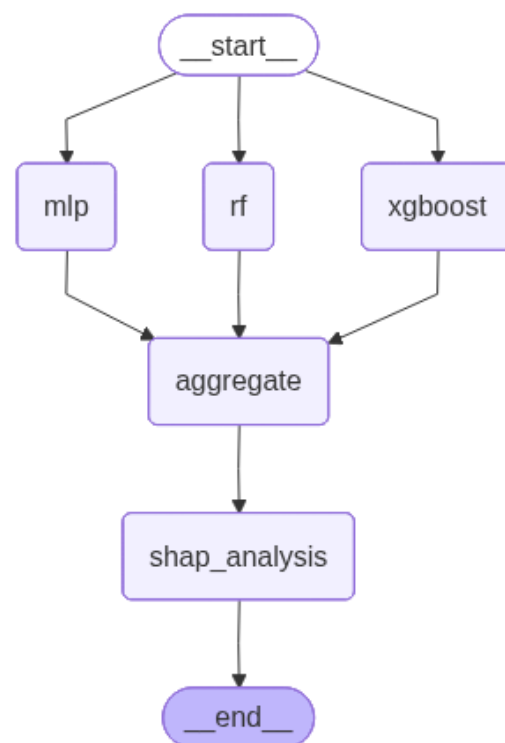


Figure 2. Ensemble subgraph flow.

3.6. Summary Agent

The final component of the pipeline is the summary agent subgraph that synthesizes classification results and SHAP analysis [7] data to create security incident reports. The agent is based on the Reasoning and Acting (ReAct) [8] pattern implemented in Lang-Graph. The agent architecture uses the llama-3.1-8b-instant for its high inference speed with tool calling and reasoning capability, with configurations shown in Table 2. The workflow follows the following stages:

Table 2. Summary agent LLM configuration.

Parameter	Value
Model	Llama-3.1-8b-instant
Provider	Groq API
Max Tokens	300
Temperature	0.0
Throughput	~560 tokens/s
Cost (Input)	\$0.05/1 M tokens
Cost (Output)	\$0.08/1 M tokens

3.6.1. Context Aggregation

In the entirety of our workflow, we maintain a universal state that holds (a) binary classification results; (b) ensemble results, including individual prediction and confidence, along with voted consensus (c) SHAP feature importance values; and (d) raw network features. The summary agent uses all of the above information as context injected into the prompt to enable context-aware generation and reduce hallucination.

3.6.2. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a technique that empowers LLM agents to use up-to-date, private knowledge beyond their original training data [39]. It also gives the model a focused context to avoid hallucination and provide relevant output. In our case, a set of three tools is provided: (a) Internal Knowledge Retrieval, (b) Tavily Web Search, and (c) ArXiv Search.

- (a) Internal Knowledge Retrieval:** A Chroma DB vector store maintains historical classification results and ingested threat intelligence documents. A Sentence Transformer model (all-MiniLM-L6-v2) is utilized to embed queries and retrieve relevant data via similarity search based on the K-Nearest-Neighbors (KNN) algorithm.
- (b) Tavily Web Search:** Tavily is a third-party service that provides Web search querying capability for autonomous workflows. Integrating Tavily allows the agent to search for the most recent information on different attack signatures and mitigation strategies.
- (c) ArXiv Search:** The ArXiv tool accesses academic literature about attack patterns and classification labels for the latest research.

3.6.3. Report Generation

The final output of the summary agent subgraph is structured as a security analysis report that provides (a) general classification, (b) attack analysis, (c) key features, (d) reasoning, and (e) mitigation recommendations.

- (a) Overall Classification:** A definitive benign/malicious verdict with a calculated confidence score and threat level (Low/Medium/High/Critical);
- (b) Attack Analysis:** Specific attack type (e.g., “DNS Flood”) and the threat vector;

- (c) **Key Features:** A list of top network features derived from the previous SHAP analysis step;
- (d) **Reasoning:** LLM’s natural language explanation of the verdict, citing evidence from retrieved context and classifier outputs;
- (e) **Mitigation Recommendations:** Actionable next steps for the incident response team. Remediation tactics such as firewall rules or protocol adjustments are supported by retrieved evidence.

With the above, the summary agent bridges the gap between raw alerts and incident-response decision-making, reducing cognitive load and moving a step towards explainable and actionable network defense.

3.7. Evaluation

To rigorously assess our graph-based workflow, we designed a framework to measure the system in the following dimensions: binary classification accuracy, individual model classification, ensemble classification, system efficiency, and the quality of the RAG system.

3.7.1. Classification Performance

Standard classification metrics are used for both binary and multi-class scenarios. **Accuracy** measures the proportion of correct predictions across all samples. **Precision** and **recall** evaluate the model’s ability to minimize false alarms and missed attacks, respectively, while the **F1 score** provides their harmonic mean as a balanced performance measure. For multi-class evaluation, the **weighted F1 score** computes a weighted average of per-class F1 scores based on the number of true instances in each class. Additionally, **confusion matrices** are used to visualize the breakdown of true positives, true negatives, false positives, and false negatives for each class.

3.7.2. RAG System Evaluation

Motivation and Challenges

In network incident-response systems, high-quality report generation is imperative, as it directly impacts security analysts’ judgment and decision-making processes. The classifier and attack detection pipeline benefits from well-established metrics based on ground-truth labels with train–test splits, but LLM-generated responses present fundamentally different evaluation challenges. Traditional metrics such as accuracy and F1 score are unsuitable for evaluating open-ended text generation because they require deterministic, categorical outputs that can be compared against known labels. LLM responses, by contrast, are non-deterministic—the same input context may produce stylistically different but semantically equivalent outputs across multiple runs due to temperature-based sampling and inherent model stochasticity.

Second, incident reports contain multiple independent factual claims that must be individually verified against source evidence. A typical security report generated by our summary agent makes 15–20 distinct assertions about attack mechanisms (e.g., “Slowloris exploits HTTP protocol”), SHAP feature attributions (e.g., “elevated Protocol field values indicate non-standard traffic”), and mitigation strategies (e.g., “implement connection time-out limits of 30 s”). Each of these claims requires independent validation—it is insufficient to evaluate the entire report as a single unit, as one fabricated claim among otherwise accurate statements could lead to dangerous security decisions. String-matching metrics like BLEU and ROUGE fail here because they measure surface-level n-gram overlap rather than factual correctness.

Third, the security-critical nature of incident response demands that recommendations be grounded in verifiable evidence rather than plausible-sounding hallucinations. In pro-

duction IDS environments, an incident responder who follows an LLM’s suggested firewall rule or protocol adjustment must trust that this advice derives from documented best practices, empirical research, or authoritative threat intelligence—not from statistical patterns in the model’s training data that happen to sound convincing. A mitigation strategy that appears authoritative but lacks empirical backing could introduce new vulnerabilities, fail to address the actual threat vector, or cause service disruptions. For example, if the LLM recommends “block all UDP traffic on port 53” without referencing DNS protocol requirements, this could prevent legitimate DNS resolution while failing to stop DNS amplification attacks that use spoofed source ports.

The large volume of information retrieved during report generation compounds these challenges. Our summary agent’s RAG workflow may retrieve 4–5 context documents from multiple sources (Tavily Web search returning recent threat intelligence, ArXiv returning academic papers on attack patterns, and internal knowledge base returning historical incident reports). Each document may contain hundreds to thousands of words. The LLM must synthesize this information, prioritize the most relevant facts, and construct a coherent narrative—a process that introduces risk. The model may inadvertently fabricate details to create narrative flow, inappropriately connect unrelated facts from different documents, or attribute conclusions to sources that never made those claims. For instance, if Document A discusses the Slowloris attack mechanics and Document B discusses connection timeout mitigation (in an unrelated context about database pooling), the LLM might incorrectly generate “Source A recommends connection timeout mitigation for Slowloris attacks”.

To address these evaluation challenges while maintaining scalability, we adopt the LLM-as-Judge framework [35] implemented through the RAGAs evaluation toolkit [36]. This approach enables automated, consistent evaluation without the prohibitive time and cost overhead of human annotation. Manual evaluation would require security experts to read each generated report (averaging 1200 characters), cross-reference it against 4–5 retrieved documents (totaling 3000–5000 words per sample), and verify each claim—a process requiring approximately 15–20 min per report. For our test set of 176 samples, this would demand hours of expert time. The LLM-as-Judge approach completes the same evaluation in approximately 45 min with consistent scoring criteria, enabling iterative prompt refinement and system optimization.

Faithfulness Metric Design

Faithfulness measures whether each atomic claim in the generated incident report can be directly verified against the retrieved context. We define an *atomic claim* as a single factual statement that can be independently evaluated for truth. For example, the sentence “DNS Flood attacks exploit UDP protocol on port 53 to overwhelm DNS servers” decomposes into four atomic claims: (1) uses UDP protocol, (2) targets port 53, (3) overwhelms DNS servers, and (4) the attack type is DNS Flood. The LLM judge evaluates each claim independently.

The evaluation uses a three-point scale:

- **Score 0 (Not Supported):** The claim is not mentioned in context or contradicts retrieved evidence. *Example:* If the context mentions “DNS attacks send UDP packets” but does not specify port 53, claiming “targets port 53” receives a score of 0.
- **Score 1 (Partially Supported):** The claim requires inference or connection of multiple fragments. *Example:* If the context states “10,000 packets per second” and the claim says “high traffic volume”, a score of 1 is assigned because “high volume” is interpretive.
- **Score 2 (Fully Supported):** The claim is explicitly stated with minimal paraphrasing. *Example:* The context states “Slowloris holds connections indefinitely using partial HTTP requests”, and the claim states “maintains persistent connections through

incomplete HTTP headers”—the context and claim are semantically equivalent despite different wording.

Only claims scoring 2 are considered faithful. The faithfulness formula is expressed as follows:

$$\text{Faithfulness}(S, C) = \frac{|\{s_i \in S : \text{score}(s_i, C) = 2\}|}{|S|}$$

where $S = \{s_1, s_2, \dots, s_n\}$ represents atomic claims extracted from the generated summary, C denotes retrieved context documents, and $\text{score}(s_i, C)$ is the LLM judge’s 0/1/2 assessment. A score of 1.0 indicates all claims are directly verifiable; scores below 0.5 warrant manual review for potential hallucinations.

Groundedness Metric Design

While faithfulness operates at the claim level with strict evidence requirements (score 2 only), groundedness evaluates semantic entailment at the sentence level and accepts scores of both 1 and 2 as grounded. A sentence is *grounded* if its overall meaning can be logically derived from context through paraphrasing or synthesis.

The three-point scale for groundedness is outlined as follows:

- **Score 0:** Sentence meaning cannot be inferred from context. *Example:* The context discusses “general DDoS mitigation”, but the generated sentence states “Deploy Cloudflare Enterprise plan”—a specific product cannot be derived from generic guidance.
- **Score 1:** The sentence requires the connection of multiple context fragments. *Example:* Context A: “Slowloris sends partial headers” + Context B: “Persistent connections exhaust resources” → Generated: “Slowloris exhausts resources by maintaining incomplete connections” (valid synthesis).
- **Score 2:** The sentence is directly derivable through paraphrasing. *Example:* Context: “maintains open connections. . . exhausting connection pool” → Generated: “exhausts server resources by holding connections” (immediate entailment).

The mathematical formulation is expressed as follows:

$$\text{Groundedness}(S, C) = \frac{\sum_{j=1}^m \mathbb{1}(\text{score}(\text{sentence}_j, C) \geq 1)}{m}$$

where m is the total number of sentences and $\mathbb{1}$ equals 1 if the sentence scores 1 or 2 (partially or fully entailed).

The complementary nature of these metrics is crucial: high groundedness with moderate faithfulness (our observed $G = 1.0$, $F = 0.64$) indicates successful synthesis while remaining evidence-bound. All sentences are derivable from context (perfect groundedness), but 36% of atomic claims involve cross-document inference rather than direct quotation (moderate faithfulness). This pattern is desirable for analytical incident reports requiring the connection of attack mechanisms to SHAP features and actionable mitigations.

Implementation Details

Both metrics are calculated using the RAGAs framework [36] with Llama-3.1-8b-instant (temperature 0.0) as the LLM judge. The prompts (Table 3) instruct the judge to evaluate support levels on the 0/1/2 scale. The evaluation completes in under an hour for 176 test samples, compared to hours for an equivalent human expert evaluation.

Table 3. Evaluation and summary agent prompts.

Prompt Type	Description
Groundedness	You are a world class expert designed to evaluate the groundedness of an assertion. You will be provided with an assertion and a context. Your task is to determine if the meaning of the assertion can be entailed from the context. <i>Rating:</i> 0 = not supported/empty, 1 = partially supported, 2 = fully supported. Provide only 0, 1, or 2.
Faithfulness	As a specialist in assessing the strength of connections between statements and their given contexts, evaluate the level of support an assertion receives from the provided context. <i>Rating:</i> 0 = not supported/empty, 1 = partially supported, 2 = fully supported. Provide only 0, 1, or 2.
Summary Agent	You are an expert security analyst generating evidence-bound security reports with access to web search (Tavily), document search, and arXiv tools. Critical rules: Every sentence must be tied to retrieved evidence; cite sources [tool_name: query_name]; state ‘Not mentioned in context’ if missing; no invented information; keep under 300 tokens. Attack types: Ping Sweep, Port Scan, ICMP/SYN/DNS/UDP/HTTP Flood, Benign. Output: OVERALL_CLASSIFICATION, CONFIDENCE, ATTACK_TYPES, PRIMARY_THREAT, THREAT_LEVEL, KEY_FEATURES, REASONING (with citations), MITIGATION_RECOMMENDATIONS (with citations), ADDITIONAL_CONTEXT, MODEL_AGREEMENT.

3.7.3. System Efficiency

Given the constraints of real-time IDSs, we consider the following computational efficiency:

- **Inference Time:** The latency of the complete agentic graph pipeline from network traffic ingestion to final summary report generation;
- **Token Usage:** The total number of tokens processed and generated with the LLM, correlating with the cost and latency of the RAG system.

3.8. Use of AI-Assisted Tools

During manuscript preparation, the authors used Claude Sonnet 4.5 and Claude Opus 4.6 (Anthropic) to assist with manuscript organization, structural editing, and refinement of written content. Grammarly (version 1.159.0.0) was used for grammar and language corrections. Writefull (cloud-integrated via Overleaf AI Assist) was used for LaTeX-specific language feedback and editing. All AI-generated suggestions were reviewed, verified, and edited by the authors.

4. Results

This section presents the quantitative evaluation of the proposed LLM-based agentic network traffic incident-report system, encompassing the classifier models’ performance and the generation quality of the RAG system. The system produces two distinct accuracy figures that should not be conflated: the binary pre-detection module (Section 4.1) performs benign-vs-malicious filtering and achieves 99.49% accuracy, while the multi-class ensemble (Section 4.2) categorizes malicious traffic into specific attack types and achieves 99.80% accuracy. The difference reflects the inherently greater difficulty of 11-class discrimination relative to binary classification, not a degradation in performance. Example full execution traces can be found in Appendix B.

4.1. Binary Pre-Detection

The lightweight Random Forest (RF) binary classifier that uses only the top-10 selected features achieved exceptional performance in distinguishing benign traffic from malicious activity. This validates and fundamentally supports our design choice of using a smaller model for early exit. The model correctly classified 232,929 out of 234,114 results, achieving an accuracy of 99.49%. The Precision is 99.80%, which means that for every 10,000 malicious

predictions, only 20 were actually benign. The recall was 99.51%, that is, 172,254 instances of malicious traffic were caught out of 173,097, with only 843 missed. The confusion matrix and the PR curve are shown in Figures 3 and 4.

The low false-negative rate of 0.49% (calculated from $1 - \text{Recall}$) is critical for network security applications, as it represents malicious traffic in the real world that could possibly evade the intrusion detection system and be incorrectly marked as benign. Figure 4 shows the precision–recall curve that represents robust performance. This stability is advantageous in various network environments that require detection of various types of attacks.

The model’s size was reduced from 37.26 MB with the full feature set to 6 MB by only including the top-10 features for edge deployment.

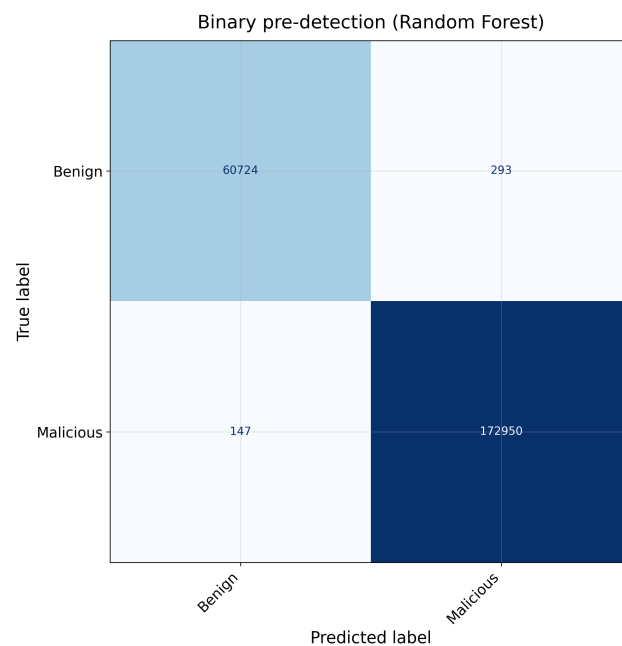


Figure 3. Binary classifier confusion matrix.

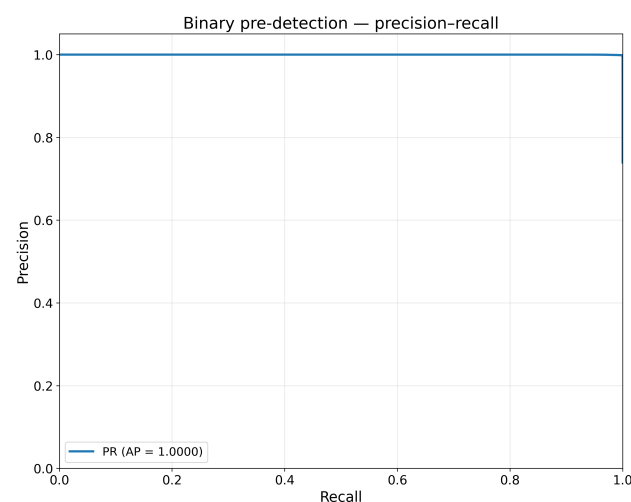


Figure 4. Binary classifier PR curve.

4.1.1. Top-10 Selected Features

Table 4 presents the top-10 features selected for the binary pre-detection classifier, ranked in descending order of Mean Decrease in Impurity (MDI) importance scores computed by the full-feature binary Random Forest model. These features represent the minimal subset that preserves near-equivalent classification accuracy while reducing the

model footprint from 37.26 MB to 6 MB. The selected features span three complementary signal categories: packet-size statistics (Packet Length Mean, Packet Length Max, Fwd Packet Length Max, Fwd Packet Length Mean, Fwd Segment Size Avg, Average Packet Size, and Subflow Fwd Bytes) that distinguish volumetric attack patterns from benign traffic; temporal flow behavior (Idle Max, Flow IAT Std), which captures timing anomalies characteristic of flooding and stealthy attacks; and protocol flag activity (ACK Flag Count), which reflects abnormal handshake behavior.

Table 4. Top-10 features selected for binary pre-detection, ranked by MDI importance.

Rank	Feature Name	Description
1	Packet Length Mean	Mean length (bytes) of all packets in the flow in both directions
2	Fwd Segment Size Avg	Average size of forward-direction TCP/UDP segments
3	Packet Length Max	Maximum packet length observed in the flow
4	Subflow Fwd Bytes	Mean bytes transferred per forward-direction subflow
5	Fwd Packet Length Max	Maximum length of forward-direction packets
6	Fwd Packet Length Mean	Mean length of forward-direction packets
7	Average Packet Size	Mean packet size across all packets in both directions
8	Idle Max	Maximum idle duration (seconds) between active flow periods
9	Flow IAT Std	Standard deviation of inter-arrival times between consecutive flow packets
10	ACK Flag Count	Count of packets with the TCP ACK flag set within the flow

4.1.2. Accuracy Calculation

Classification accuracy is computed as the proportion of correctly classified samples among the total number of test samples. In the implementation, scikit-learn's `accuracy_score` function is applied to the held-out test set following the 80/20 train-test split. Formally:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{Number of Correct Predictions}}{\text{Total Test Samples}}$$

For the binary classifier, the test set comprises 234,114 samples; of these, 232,929 were correctly classified (both benign traffic correctly identified as benign and malicious traffic correctly identified as malicious), yielding an accuracy of $232,929/234,114 = 99.49\%$. The same formulation is applied to the multi-class ensemble classifiers, where the numerator accumulates all correctly predicted samples across all 11 attack classes and the denominator is the total multi-class test partition size.

4.2. Multi-Class Attack Classification

Table 5 presents a performance comparison of three classifier models of the ensemble subgraph—Random Forest (RF), XGBoost, and Multi-Layer Perceptron (MLP) models—and the results of the ensemble.

The Random Forest classifier achieves the best precision of 99.8214%, a weighted F1 score of 99.8174%, and a macro F1 score of 99.2632%.

The Multi-Layer Perceptron (MLP) model shows a notable divergence between its macro F1 score (93.49%) and weighted F1 score (99.23%). The weighted F-score computes each class's F1 score averaged and weighted by class support (number of true instances), so high-frequency classes such as Port Scan (88,126 samples) and ICMP Flood (45,047 samples) dominate the aggregate. The macro F1 score, by contrast, weights all classes equally, regardless of support, exposing weaknesses in minority classes. The 5.74 percentage-point gap between these two metrics for MLP reveals that its strong performance on the majority classes masks poor detection of under-represented attack types. This is exemplified in the per-class F1 score (Table 6), with MLP achieving only 0.4828 on UDP Flood (158 samples),

dragging down the unweighted macro average. In contrast, Random Forest shows only a 0.56 percentage-point gap (99.8174% weighted vs. 99.2632% macro), indicating more uniform performance across all classes. This divergence pattern underscores why macro F1 is the more informative metric for security applications where detecting rare attack types is critical, and it further validates the use of the ensemble approach to compensate for individual model weaknesses on minority classes. Notably, the MLP model has its advantages in size. In particular, the MLP model is the smallest in terms of model size, making it storage-efficient in edge environments.

The XGBoost model has the most balanced metrics in terms of accuracy, weighted F1 score, and macro F1 score and has a model size of 1.77 MB, making it only slightly larger than MLP, but maintains a relatively consistent prediction result across all attack classes.

A key observation of the ensemble model is that it matches XGBoost's weighted F1 score of 99.80% while increasing the Macro F1 score to 99.23% as noted in Table 5. This shows the marginal gains achieved by combining multiple models, even when their accuracy is similar. The ensemble model exemplifies its ability in error correction, as evidenced by Figures 5–7.

The apparent similarity of weighted metrics across Random Forest, XGBoost, and the Ensemble models shown in Table 5 warrants explanation. This convergence is a mathematical property of support-weighted averaging on this dataset, not evidence of overfitting. Three dominant classes—Port Scan (88,155 samples, 37.7% of the test set), Benign (61,017, 26.1%), and ICMP Flood (45,047, 19.2%)—collectively account for 83.0% of the test partition. Random Forest, XGBoost, and the Ensemble model achieve near-perfect per-class F1 scores on these classes (>0.996 , as shown in Table 6), while even MLP exceeds 0.987 on all three, causing the support-weighted aggregates to converge toward similar values. The Macro F1 metric, which assigns equal weight to all 11 classes, regardless of sample count, reveals the true inter-model differentiation: Random Forest, 99.2632%; XGBoost, 99.0851%; MLP, 93.4902%; and Ensemble, 99.2284%. Reporting metrics at four decimal places further exposes the variation masked by two-decimal rounding. Additionally, 10-fold stratified cross-validation on the training set confirms stable generalization: Random Forest achieves a mean weighted F1 of 0.9980 ± 0.0001 across folds, and MLP achieves 0.9911 ± 0.0012 . The close alignment between cross-validation means and test-set performance (RF: CV 0.9980 vs. test 0.9982; MLP: CV 0.9911 vs. test 0.9923) confirms that the models generalize well rather than overfitting to the training partition.

From Table 5, it is evident that the Random Forest model beats the Ensemble model in all metrics. Arguably, we could have only included the Random Forest model, as it generally performs better. However, ensemble voting offers an opportunity for correction of errors and identification of ambiguous cases. Each model has its own misclassifications, while the soft-voting mechanism smooths outliers and makes more reliable predictions in marginal cases. This is crucial because, for an automated incident-response system, the classified attack class directly impacts what information is relevant. In addition, for an intrusion detection system, disagreement among models signifies uncertainty and presents an opportunity for further investigation and human intervention.

Figure 5 presents the confusion matrix for the XGBoost classifier. We noticed that the error pattern of the XGBoost model is clean. Almost all misses go to “Benign”, and it rarely confuses across attack types. This is beneficial because it shows that XGBoost is a strong individual model for identifying the correct attack types and guiding the majority of the ensemble toward the correct prediction labels.

Figure 6 presents the confusion matrix of the MLP classifier, showing an inconsistent error profile, as there were multiple misclassifications between attack types. For example, there were 168 OS Scan records misclassified as Port Scan, showing confusion between

scanning attacks. There were also 41 Ping Sweep instances misclassified as OS Scan, which represents confusion in the same scanning family, but XGBoost did not have this problem. A total of 598 Benign records were also marked as false-positive SYN Flood instances, which is unique to the MLP model. Moreover, MLP performed worst in the UDP Flood class, with only 56/158 instances correctly classified (35.4% recall), compared to XGBoost's 147/158 (93.0%).

As observed through its confusion matrix in Figure 5, XGBoost is generally a strong individual model. The ensemble is worse than XGBoost alone at handling false positives because the MLP model has a high false-positive rate. But the Ensemble outperforms XGBoost with respect to Ping Sweep attack types, reducing the number of errors from 32 to 2 through soft voting.

Table 5. Comparison of model performance. Metrics are reported to four decimal places to reveal inter-model variation masked by rounding at high accuracy levels. Macro F1, which weights all classes equally, shows the most differentiation. Bold values indicate the best result per metric.

Metric	Random Forest	XGBoost	MLP	Ensemble (Vote)
Accuracy	99.8163%	99.8001%	99.2264%	99.7971%
Precision	99.8214%	99.8038%	99.2673%	99.8029%
Recall	99.8163%	99.8001%	99.2264%	99.7971%
Weighted F1	99.8174%	99.8010%	99.2285%	99.7983%
Macro F1	99.2632%	99.0851%	93.4902%	99.2284%
Model Size	151.17 MB	1.77 MB	0.50 MB	—

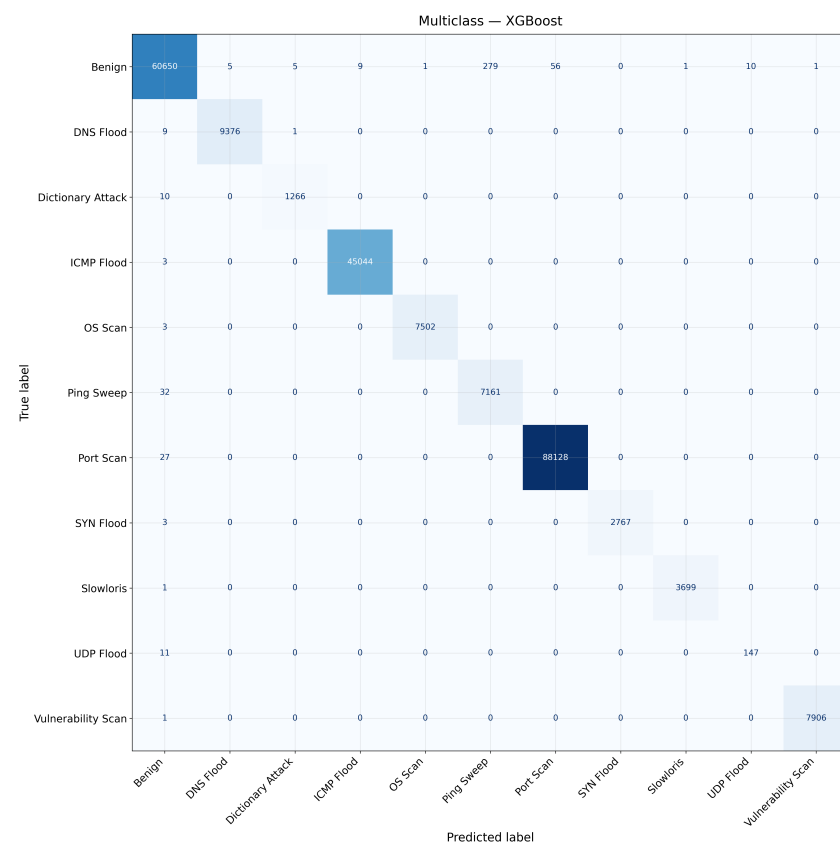


Figure 5. XGBoost confusion matrix.

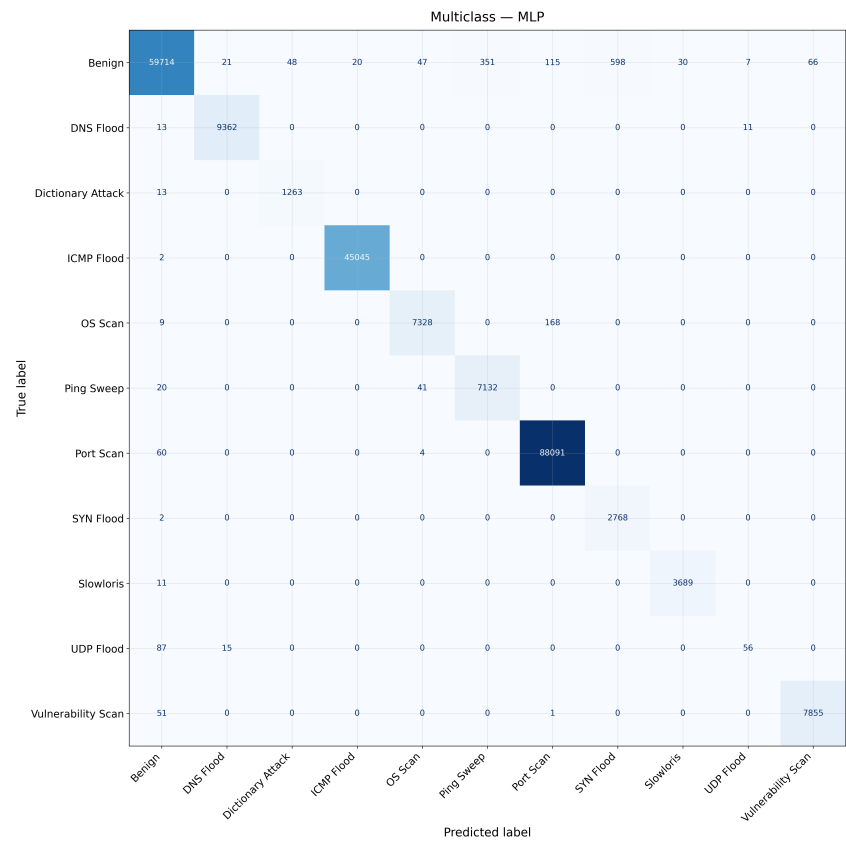


Figure 6. MLP confusion matrix.

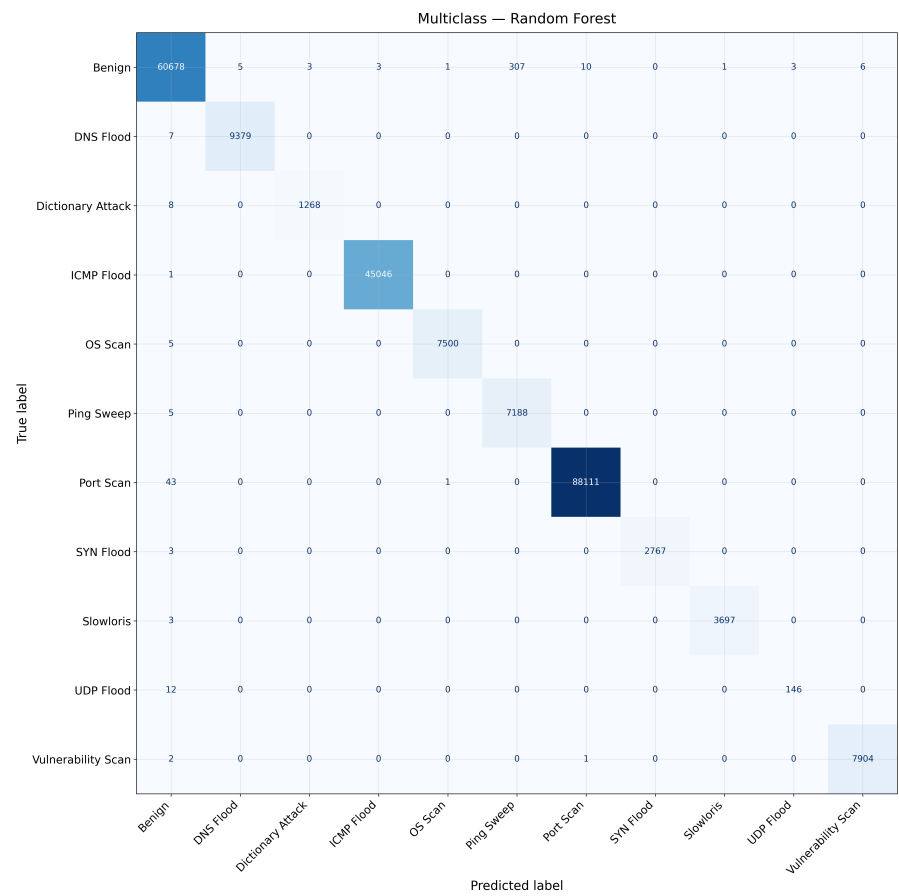


Figure 7. Multiclass RF confusion matrix.

Figure 7 presents the confusion matrix for the Random Forest multiclass classifier, demonstrating the model's strong individual performance that contributes to ensemble reliability. The Random Forest model achieves exceptionally clean diagonal values, with minimal cross-class confusion. Unlike the MLP model, which exhibits scattered errors across multiple attack types, Random Forest maintains a consistent error pattern where nearly all misclassifications collapse to the Benign class rather than exhibiting confusion between attack types.

The model demonstrates near-perfect performance on several attack classes. ICMP Flood achieves 45,046 correct classifications, with only a single false negative misclassified as Benign, representing 99.998% accuracy for this class. Similarly, Slowloris (3697/3700 correct) and SYN Flood (2767/2770 correct) exhibit exceptional detection rates. Port Scan shows robust performance, with 88,111 correct classifications, though 43 samples were misclassified as Benign and 1 as OS Scan—the latter representing one of the few cross-attack confusions in the RF model.

The UDP Flood class, which severely challenges the MLP model (35.4% recall), demonstrates significantly improved performance under Random Forest, achieving 146/158 correct classifications (92.4% recall). This 57-percentage-point improvement over MLP highlights Random Forest's superior handling of minority classes with ambiguous feature patterns. The 12 misclassified UDP Flood samples all collapse to Benign rather than other attack types, suggesting the model errs conservatively when uncertain rather than producing false attack-type associations.

Benign traffic classification reveals the model's conservative bias, with 60,678 correct predictions but notable false positives across multiple attack categories. The largest false-positive cluster appears in Ping Sweep class (307 benign samples misclassified as malicious), followed by Port Scan (10), Vulnerability Scan (6), DNS Flood (5), and several smaller clusters: Dictionary Attack (3), ICMP Flood (3), UDP Flood (3), OS Scan (1), and Slowloris (1). This pattern indicates that Random Forest occasionally flags legitimate reconnaissance-like network behavior or DNS queries as potentially malicious, prioritizing sensitivity over specificity to minimize the risk of undetected attacks in operational environments.

Figure 8 reveals the ensemble's error-correction capabilities through soft voting aggregation of the three constituent models. Comparing the ensemble confusion matrix to the individual Random Forest matrix (Figure 7) demonstrates targeted improvements in specific attack classes where individual models exhibit weaknesses. The Ensemble model achieves 60,615 correct Benign classifications, compared to Random Forest's 60,678, representing a marginal 63-sample decrease. However, this trade-off enables substantial improvements in challenging attack classes through voting-based error correction.

Port Scan demonstrates the most significant ensemble benefit, improving from 88,111 correct classifications (RF alone) to 88,126 (+15 samples), while simultaneously reducing false positives from 43 to 28 (−15 samples). This bidirectional improvement occurs because XGBoost and MLP occasionally correct Random Forest's conservative misclassification of benign traffic as Port Scan, while RF corrects the other models' confusion of Port Scan with other reconnaissance attacks. Similarly, Ping Sweep improves from 7188 (RF) to 7191 (+3), Slowloris from 3697 to 3698 (+1), UDP Flood from 146 to 147 (+1), and Vulnerability Scan from 7904 to 7906 (+2). These incremental gains validate the ensemble's ability to recover from individual model errors through complementary decision boundaries.

The ensemble's value becomes particularly evident when comparing to the MLP model's catastrophic failure on UDP Flood (56/158 correct, 35.4% recall). While Random Forest achieves 92.4% recall and XGBoost achieves 93.0% recall on this class, the ensemble maintains 93.0% recall (147/158) by leveraging the strengths of RF and XGBoost to overcome MLP's weakness. The voting mechanism effectively filters out MLP's unreliable

UDP Flood predictions while preserving its contributions to classes where it performs competitively, such as Port Scan (F1: 0.998) and ICMP Flood (F1: 0.9998).

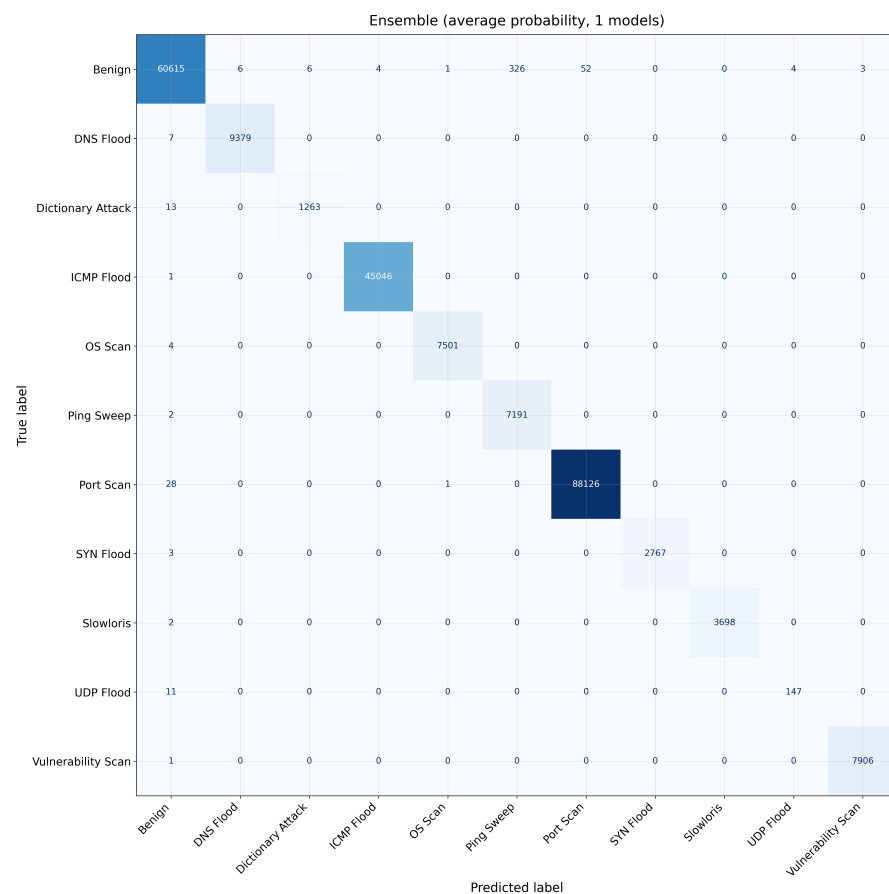


Figure 8. Ensemble confusion matrix.

Dictionary Attack presents an interesting counterexample where the ensemble (1263 correct) marginally underperforms Random Forest alone (1268 correct, −5 samples). This degradation occurs when MLP and XGBoost occasionally misclassify Dictionary Attack samples that RF correctly identifies, and the 2-to-1 voting ratio overrides RF's correct prediction. However, the 0.4% accuracy loss on this single class ($1263/1276 = 98.98\%$ vs. 99.37%) represents an acceptable trade-off for the ensemble's overall robustness across all 11 classes, particularly given the substantial improvements in Port Scan (+15) and the critical preservation of high performance on security-critical classes like DNS Flood (F1: 0.9993) and Slowloris (F1: 0.9997).

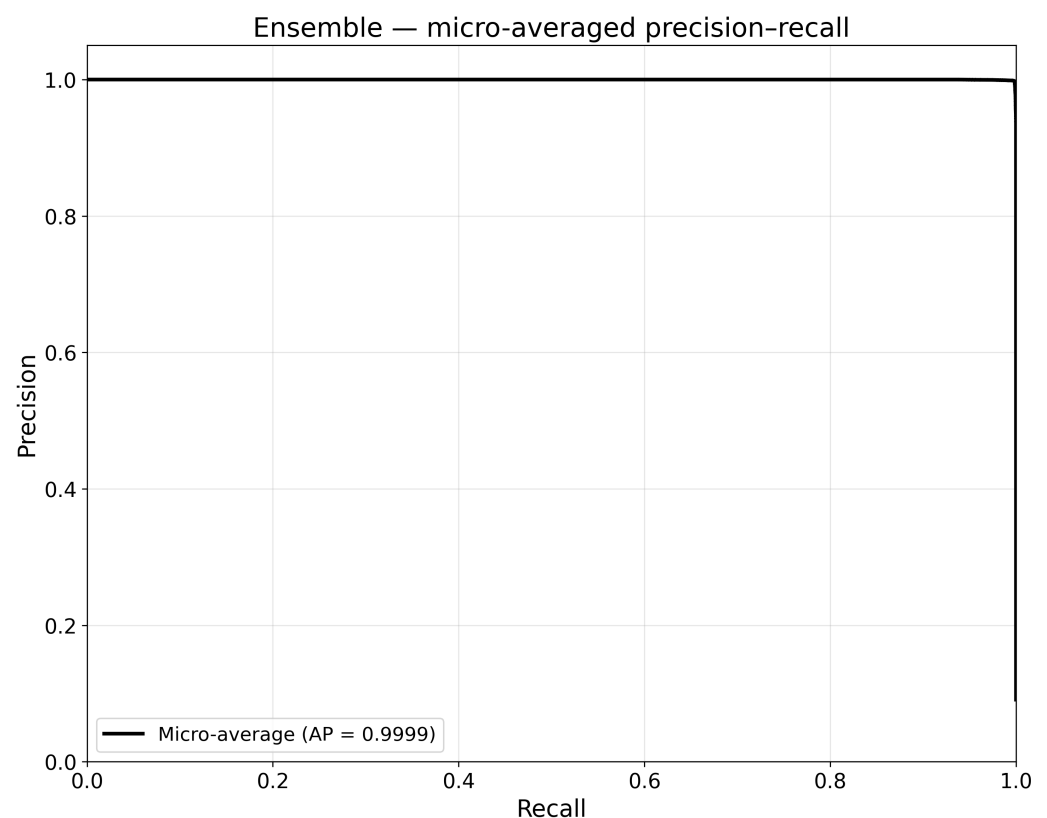
Table 6 presents a comparison of the F1 score per class between different models and attack types, while Figure 9 shows the precision–recall curve of the ensemble predictions. Among the attack classes, ICMP flood achieves the best detection, with the Random Forest model scoring a perfect 1.000 and the overall result of the ensemble reaching 0.999. This high accuracy comes from the consistent attack pattern in the ICMP Flood class that generates high-frequency ICMP echo requests. Similarly, Slowloris (ensemble F1: 0.9997) and Vulnerability Scan (0.9997) exhibit consistent feature patterns that all three models identify reliably. DNS Flood demonstrates the value of ensemble aggregation, as it improves the individual model's 0.9968–0.9994 to an ensemble F1 score of 0.9993.

Table 6. Per-class F1-score comparison. Bold values indicate the best result per class.

Attack Class	Random Forest	XGBoost	MLP	Ensemble
Benign	0.9965	0.9962	0.9870	0.9961
DNS Flood	0.9994	0.9992	0.9968	0.9993
Dictionary Attack	0.9957	0.9937	0.9764	0.9925
ICMP Flood	1.0000	0.9999	0.9998	0.9999
OS Scan	0.9995	0.9997	0.9820	0.9996
Ping Sweep	0.9788	0.9787	0.9719	0.9777
Port Scan	0.9997	0.9995	0.9980	0.9995
SYN Flood	0.9995	0.9995	0.9022	0.9995
Slowloris	0.9995	0.9997	0.9945	0.9997
UDP Flood	0.9511	0.9333	0.4828	0.9515
Vulnerability Scan	0.9994	0.9999	0.9925	0.9997

The UDP Flood attack class remains a challenging outlier among the models and exemplifies the importance of an ensemble approach. The nature of UDP packet patterns and the imbalance in the training dataset create a challenge for the MLP model to correctly identify. Legitimate UDP applications may present packet patterns similar to a UDP Flood attack, further challenging the classification.

Reconnaissance attacks, including Ping Sweep, Port Scan, and OS Scan, show moderate classification difficulty, but the present confusion challenges the classifier models. Ping Sweep achieves the second lowest ensemble F1 score of 0.9777, with all three models performing similarly (0.9719–0.9788). The MLP confusion matrix in Figure 6 reveals that Ping Sweep is often misclassified as benign samples, and 41 of the OS Scan samples were falsely predicted as Ping Sweep. This shows confusion among reconnaissance attacks as one of MLP model's limitations.

**Figure 9.** Ensemble PR curve.

4.3. Summary Agent Evaluation

The performance of the summary agent was evaluated with the RAGAs framework [36] on a test set of 176 samples (from 200 evaluation attempts with 24 errors excluded). We analyze the faithfulness and groundedness of the responses using the LLM-as-Judge framework [35], the results of which are shown in Table 7.

The groundedness of the response achieved a perfect 1.0 score, which means that all generated claims successfully utilized the context of retrieval. Individual sentences are rated on a 0/1/2 scale (not supported/partially supported/fully supported), then aggregated as the proportion of grounded sentences (score ≥ 1), yielding a final metric range of [0, 1]. The faithfulness score of 0.6391 indicates that approximately 64% of atomic claims were directly verifiable with minimal paraphrasing (score = 2), while the remaining claims required inference across multiple sources. This pattern reflects appropriate analytical synthesis rather than verbatim quotation of retrieved documents.

Table 7. Summary agent evaluation metrics. Individual assessments use a 0/1/2 scale, aggregated into final scores ranging from 0 to 1. Bold indicates the best score per metric.

Metric	Score	Output Range	Judge LLM
Groundedness	1.0000	[0, 1]	llama-3.1-8b-instant
Faithfulness	0.6391	[0, 1]	llama-3.1-8b-instant

4.3.1. Context and Response Length Analysis

Figure 10 presents two critical relationships that reveal the retrieval and synthesis behaviors of the summary agent. The left panel plots total retrieved contexts (from Tavily, ArXiv, and internal knowledge base) against generated response length, while the right panel correlates response length with the two RAG quality metrics.

The left panel demonstrates that the summary agent retrieves four to five context documents for the majority of samples, with this mode accounting for approximately 70% of the test set. However, context volume exhibits substantial variance, ranging from 4 contexts (minimal retrieval) to 16 contexts (extensive research). Despite substantial variation in retrieved context volume (ranging from 4 to 16 documents), response lengths remain remarkably consistent, clustering between 1150 and 1600 characters. Samples utilizing four to five contexts (the dominant mode) produce responses spanning nearly the full length range, as do samples with 8, 12, or 16 contexts. This pattern indicates that the agent synthesizes variable information volumes into standardized report formats rather than scaling output length proportionally to input volume. The consistency reflects the prompt constraint specifying a maximum of 300 tokens, which the agent respects while adapting content depth to available evidence rather than padding with redundant information when fewer contexts are retrieved.

The right panel reveals a striking divergence between the two RAG metrics across response lengths. Groundedness (red circles) maintains perfect 1.0 scores universally—every single sentence across all 176 test samples, regardless of response length (1150–1600 characters), can be traced to the retrieved context through entailment. This zero-variance groundedness demonstrates exceptional retrieval coverage and agent discipline in avoiding hallucinated content. The agent successfully resists the temptation to “fill space” with plausible-sounding but unverified claims when approaching the 300-token limit.

In contrast, faithfulness (green triangles) exhibits substantial variance, ranging from approximately 0.1 to 0.9 across response lengths. Samples of similar response lengths (e.g., 1300–1400 characters) display faithfulness scores spanning the full range from 0.3 to 0.9, revealing inconsistency in how the agent handles source material. This variance indicates a limitation in controlling synthesis depth: some responses closely paraphrase retrieved

documents (higher faithfulness), while others make claims requiring extensive inference or connecting fragments across multiple sources (lower faithfulness). The low-faithfulness cases ($F < 0.4$), though still maintaining perfect groundedness ($G = 1.0$), suggest the agent occasionally over-synthesizes by combining information in ways such that individual atomic claims cannot be directly verified against the source text. This inconsistency highlights the challenge of maintaining uniform evidence standards across diverse attack types and retrieval contexts, warranting future work on prompt engineering or fine-tuning to achieve more predictable faithfulness levels while preserving analytical depth.

Figure 11 shows the distribution of response lengths across the 176-sample test set, with a mean of 1384 characters. The distribution shows concentrated frequency around 1300–1450 characters, with notable peaks near 1400 characters. The majority of responses fall between 1200 and 1500 characters, demonstrating the agent’s adherence to the 300-token constraint while adapting content depth to the specific incident scenario.

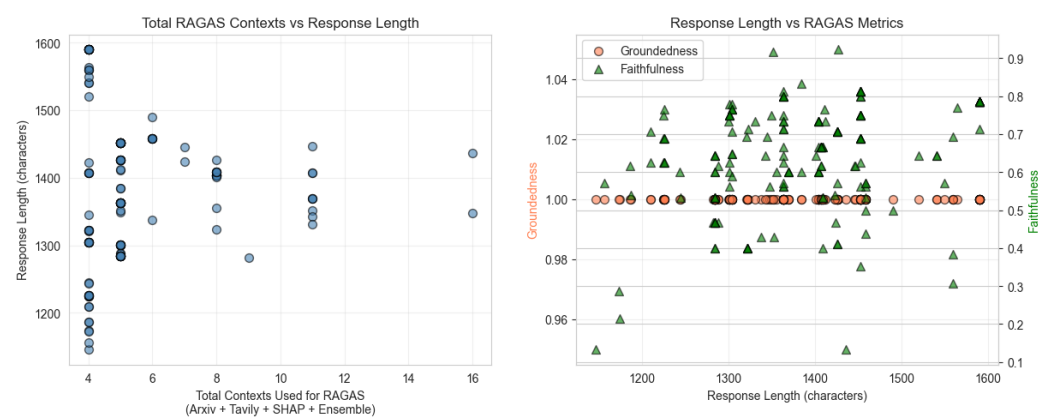


Figure 10. Total contexts vs. response length (left) and response length vs. RAG metrics (right). Blue points represent groundedness scores; orange points represent faithfulness scores.

Figure 11 shows the distribution of response lengths across the test set.

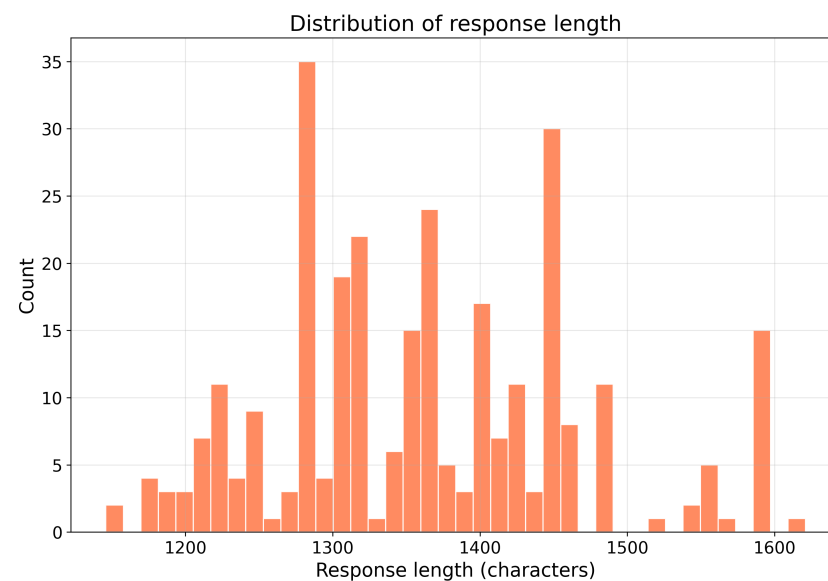


Figure 11. Distribution of response lengths.

4.3.2. Groundedness and Faithfulness Distribution Analysis

Figures 12 and 13 present the distributions of groundedness and faithfulness scores across 176 evaluation instances (from 200 evaluation attempts with 24 errors ex-

cluded), revealing fundamentally different quality patterns that validate our dual-metric evaluation framework.

Figure 12 demonstrates perfect groundedness consistency: 100% of generated reports achieve the maximum score of 1.0, with zero variance across all samples. This exceptional result indicates that every sentence in every generated incident report—spanning diverse attack types from DNS Flood to Slowloris and benign traffic—derives its semantic meaning from retrieved context documents. The LLM judge identified no instances of sentence-level hallucination where overall meaning could not be traced to Tavily search results, ArXiv papers, or internal knowledge-base entries. This perfect score validates several critical design decisions: (1) the prompt-engineering strategy explicitly prohibiting unverified claims (“state ‘Not mentioned in context’ if missing; no invented information”); (2) the temperature setting of 0.0 ensuring deterministic, conservative generation; and (3) the RAG architecture’s comprehensive retrieval coverage across three complementary sources (real-time Web intelligence, scholarly literature, and historical incident data).

The groundedness result also confirms that the agent successfully resists common failure modes observed in production LLM systems. There is no evidence of “narrative fabrication” where the model invents plausible-sounding technical details to create coherent flow, “source hallucination” where it attributes claims to documents that never made those statements, or “confidence extrapolation” where it presents knowledge derived from training data as if it were retrieved evidence. The perfect groundedness score across attack types ranging from common (Port Scan: 88,126 samples in ensemble) to rare (UDP Flood: 147 samples in ensemble) demonstrates that retrieval coverage remains adequate, even for minority classes where Web search results may be limited.

In stark contrast, Figure 13 reveals substantial faithfulness variance with a left-skewed distribution (mean: 0.6391; median: 0.67; std: 0.14). The distribution shows concentration in the 0.60–0.75 range, with a tail extending to lower scores. This variance is not a system failure but, rather, evidence of the agent’s adaptive synthesis capability. Faithfulness measures whether atomic claims are *directly* verifiable with minimal paraphrasing (score of 2), whereas groundedness measures whether sentence meanings are *derivable* through inference or cross-document connection (scores of 1–2 acceptable).

The faithfulness distribution reveals distinct report patterns based on synthesis depth. High-faithfulness reports correspond to well-documented attack types (ICMP Flood, DNS Flood, and SYN Flood) where retrieved threat intelligence directly states most claims, requiring minimal synthesis. The agent predominantly paraphrases source material, achieving claim-level verifiability but limited analytical value-add. Moderate-faithfulness reports represent the intended operational mode: the agent connects attack signatures from Tavily to SHAP feature explanations, derives security implications, and synthesizes mitigation strategies by combining information across documents. For example, a moderate-faithfulness report might state “The elevated Protocol field values (SHAP rank 1) indicate Slowloris attack behavior, which exhausts server resources through persistent incomplete HTTP connections”. This atomic claim decomposes into (1) “Protocol field elevated” (SHAP output), (2) “indicates Slowloris” (Tavily web search), (3) “exhausts server resources” (ArXiv paper), and (4) “persistent incomplete HTTP connections” (Wikipedia retrieved via Tavily). While the overall sentence is grounded ($G = 1.0$), individual claims require the connection of three to four separate sources, reducing faithfulness.

Low-faithfulness cases warrant investigation but do not necessarily indicate failure. These cases typically involve complex, multi-stage attacks (e.g., reconnaissance followed by exploitation) where the agent constructs causal chains (“Port Scan detection suggests preparation for targeted intrusion”) that require extensive inference. The fact that these reports maintain $G = 1.0$ indicates that the reasoning remains evidence-based at the

sentence level, even if individual atomic claims cannot be directly quoted. However, extremely low faithfulness scores merit manual review to verify the appropriateness of inferential leaps, as they may indicate over-synthesis bordering on speculation.

The complementary nature of the two metrics becomes evident: the perfect groundedness ($G = 1.0$) establishes a safety floor ensuring no hallucinated content, while the moderate faithfulness ($F = 0.64$, std: 0.14) confirms that the agent performs meaningful synthesis rather than mere paraphrasing. A hypothetical system achieving $G = 1.0$ and $F = 1.0$ would simply quote retrieved documents verbatim, providing no analytical value to security analysts who could read the source material directly. Conversely, $G < 0.8$ with any faithfulness score would indicate problematic hallucinations. Our observed pattern ($G = 1.0$, $F = 0.64$) represents a good balance for analytical incident reporting.

Figure 14 visualizes the relationship between groundedness and faithfulness across 176 evaluation instances (from 200 evaluation attempts with 24 errors excluded), revealing a striking independence between the two metrics. Groundedness forms a perfect vertical line at $x = 1.0$, while faithfulness scatters vertically across a wide range with no horizontal variance.

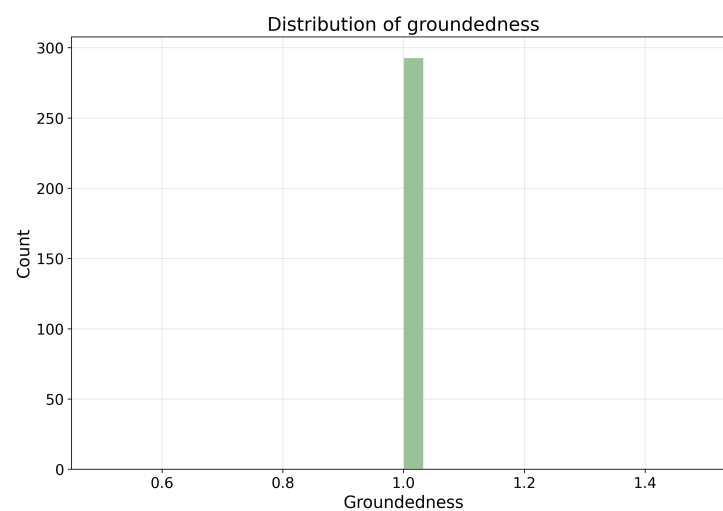


Figure 12. Distribution of groundedness.

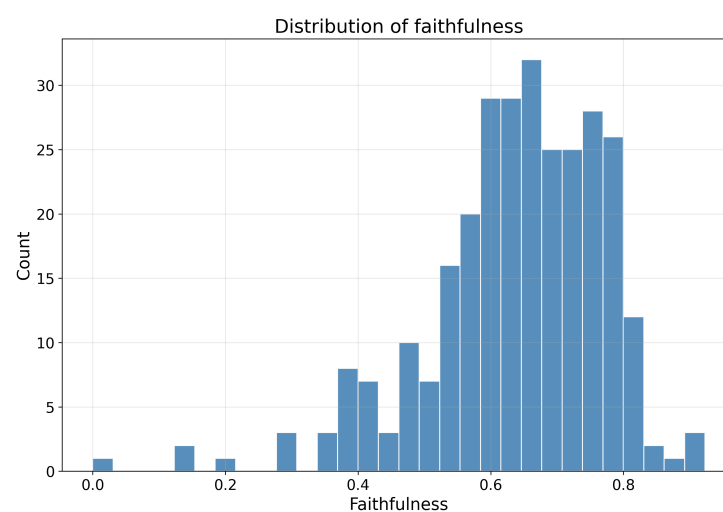


Figure 13. Distribution of faithfulness.

This vertical distribution pattern—zero groundedness variance with substantial faithfulness variance—confirms that the metrics measure fundamentally orthogonal quality

dimensions. Groundedness evaluates sentence-level semantic entailment: “Can this sentence’s meaning be derived from context?” This is a binary pass/fail criterion at the sentence level, aggregated across all sentences. Faithfulness evaluates claim-level direct verifiability: “Can this atomic fact be quoted from context with minimal paraphrasing?” This is a more granular, stricter criterion requiring explicit textual support rather than inferential entailment.

The lack of correlation demonstrates that high groundedness does not predict high faithfulness, and vice versa. A sentence like “Slowloris exhausts server connection pools by maintaining incomplete HTTP requests indefinitely” achieves perfect groundedness (score 2) because the overall meaning is derivable by combining Source A (“Slowloris sends partial HTTP headers”) and Source B (“persistent connections exhaust server resources”). However, the individual atomic claims—“exhausts connection pools,” “incomplete HTTP requests,” and “indefinitely”—each requires paraphrasing or inference from sources that use different terminology (“server limits,” “partial headers,” and “holds connections open”), resulting in faithfulness scores of 1 (partially supported) rather than 2 (fully supported). Thus, the sentence contributes 1.0 to groundedness (score 2 out of 2) but only 0.33 to faithfulness (assuming one of three claims score 2 and the others score 1).

The distribution also reveals no faithfulness “ceiling effect” despite perfect groundedness. If groundedness were simply a looser version of faithfulness, we would expect faithfulness scores to cluster near 1.0 when groundedness reaches 1.0. Instead, faithfulness is distributed with a slight left skew around the mean of 0.64, indicating the agent freely synthesizes information (reducing faithfulness) while maintaining semantic grounding (preserving groundedness). This independence validates our dual-metric framework: evaluating with faithfulness alone would penalize desirable analytical synthesis, while evaluating with groundedness alone would fail to detect over-inference where agents make logical leaps beyond what sources explicitly state.

The vertical scatter pattern at a constant of $G = 1.0$ with faithfulness variance (mean: 0.64; std: 0.14) provides operational guidance for system tuning. Increasing temperature or reducing the “cite sources” prompt constraint would shift the distribution leftward (lower faithfulness) while potentially breaking the $G = 1.0$ floor. Conversely, prompts demanding “quote sources directly” would shift rightward (higher faithfulness) but reduce analytical value. The current configuration represents the optimal operating point for security incident analysis, requiring evidence-based synthesis rather than document summarization.

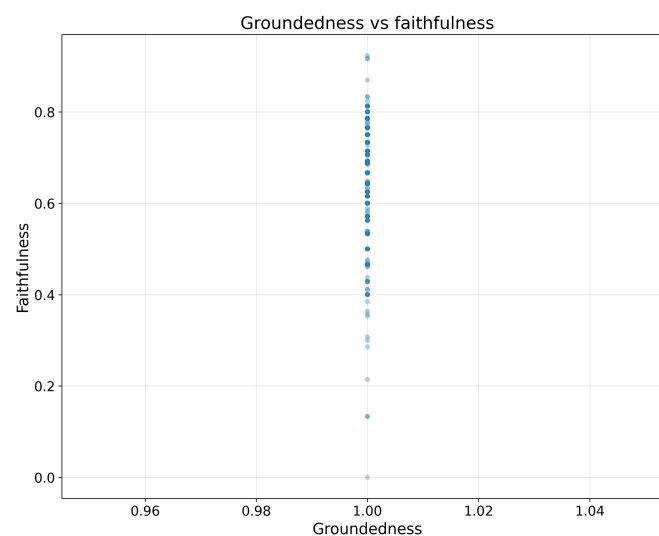


Figure 14. Groundedness vs. faithfulness across 176 evaluation instances. Each point represents one generated incident report; the vertical cluster at groundedness = 1.0 reflects perfect grounding, while faithfulness varies along the y-axis.

4.4. Operational Efficiency

We measure the efficiency using the following metrics:

- **Inference Speed:** Ranges between 2.9 and 11.6 s per sample, including model inference, external tool calls, and generation on malicious traffic.
- **Tokens:** The average number of completion tokens is 557 per malicious sample. The average number of prompt tokens is 4689 per malicious sample. These sum up to 5247 tokens.

Summary of Key Findings

The experimental results demonstrate that the proposed graph-based agentic framework successfully integrates high-accuracy classification with explainable, evidence-grounded incident reporting.

- **High Classification Accuracy:** The system achieves detection rates exceeding 99.8% across all classification tasks. The lightweight Random Forest pre-detection module (99.49% accuracy, 6 MB model size) enables efficient filtering suitable for edge deployment.
- **Evidence-Grounded Report Generation:** The RAG-based summary agent achieves perfect groundedness scores (1.0), ensuring all generated claims derive from retrieved threat intelligence. Moderate faithfulness (0.64) reflects appropriate analytical synthesis rather than verbatim paraphrasing.
- **Explainability Through SHAP and Natural Language:** SHAP analysis identifies influential network features for each classification decision. Natural language explanations citing retrieved evidence enable security analysts to verify automated recommendations, addressing trust gaps in black-box ML systems.
- **Ensemble Resilience:** Soft voting provides robustness to individual model weaknesses, improving the UDP Flood F1 score from 48% (MLP alone) to 95% through ensemble aggregation.

4.5. Discussion of Results

The pre-detection module demonstrates that a compact 6 MB Random Forest model restricted to 10 features can serve as an effective gatekeeper, achieving 99.49% accuracy and dropping the majority of benign traffic before the computationally intensive ensemble and RAG stages are invoked. The initial pre-detection scheme rendered only 0.3 s latency in our experimental environment, confirming its suitability for time-sensitive filtering.

The ensemble approach proves particularly valuable for challenging minority attack classes. By aggregating soft-vote probabilities from MLP, Random Forest, and XGBoost, the framework compensates for individual model weaknesses—most notably, improving the UDP Flood F1 score from 48% (MLP alone) to 95% while preserving strong performance on well-represented classes. The perfect groundedness score of the summary agent ($G = 1.0$) confirms that all generated claims are grounded in retrieved context and SHAP analysis results, consistent with prior research by Asal et al. and Wang et al. [23,40], who demonstrated that evidence-based explanation mechanisms enhance trustworthiness in AI-based IDSs.

The faithfulness score of 0.64 warrants careful interpretation in the context of operational deployment. A score above 0.5 indicates that the majority of atomic claims in the generated report are directly verifiable against the retrieved context, while a perfect 1.0 would require every statement to be traceable to a source document with minimal paraphrasing. Our score of 0.64 suggests that roughly two-thirds of the report content is faithfully grounded, while the remaining third involves cross-document inference or analytical synthesis not directly quotable from any single source. For operational deployment,

this implies that generated incident reports should be treated as *analyst-assistive* drafts rather than fully autonomous outputs—a human-in-the-loop review remains advisable before incident reports are disseminated to response teams. Improvement of faithfulness through retrieval refinement (e.g., more granular chunking strategies or domain-specific re-ranking) is identified as a priority for future work.

To contextualize this result, recent surveys on RAG evaluation have established that faithfulness—measured as the proportion of generated claims entailed by the retrieved context—is a core quality dimension alongside relevance and completeness [36,41,42]. Reported faithfulness scores in current RAG systems typically range from approximately 0.6 to 0.9 depending on task complexity, domain specificity, and model capability, with scores below 0.6 generally considered unreliable for high-stakes applications and scores above 0.8 regarded as strong [41,43]. Chen et al. further demonstrated that even state-of-the-art LLMs exhibit substantial weaknesses in RAG-critical abilities such as noise robustness, information integration across multiple documents, and counterfactual robustness [33]. Our domain-specific cybersecurity setting—where cross-document synthesis of SHAP explanations, threat intelligence, and classification metadata are required—represents an especially challenging context for faithful generation. Thus, the 0.64 score falls in the moderate range, consistent with systems that benefit from human oversight, while highlighting clear headroom for improvement through domain-adapted retrieval and generation strategies, as discussed in recent RAG evaluation surveys [34].

4.6. Limitations and Future Work

Despite its strong performance, the proposed framework presents several limitations that motivate future research directions:

- **End-to-end latency:** The complete pipeline, from traffic ingestion to final report generation, exhibits inference times ranging from 2.9 to 11.6 s per malicious sample, driven primarily by external tool calls in the summary agent. This range may not meet the sub-second requirements of production IDS environments. Future work should investigate response caching for repeated attack signatures, model distillation to replace the Llama-3.1-8b-instant model with a smaller fine-tuned variant, and asynchronous tool-call execution to reduce wall-clock latency.
- **SHAP feature attribution ablation:** The current system applies SHAP analysis [7] to RF and XGBoost only, as TreeExplainer does not natively support the MLP architecture. Furthermore, the contribution of SHAP-derived features as inputs to the summary agent prompt has not been formally ablated. A systematic ablation study removing SHAP context from the LLM prompt and evaluating the resulting changes in faithfulness and groundedness scores would quantify the interpretability contribution of SHAP to report quality. This is consistent with XAI evaluation approaches proposed in prior work [21,22].
- **RAG component ablation.** The system has not been benchmarked in a RAG-ablated configuration where external retrieval (Tavily, ArXiv, and internal knowledge base) is disabled and the LLM generates reports from the classification context alone. Such an ablation would isolate the contribution of retrieval to groundedness and faithfulness scores, providing empirical justification for the added infrastructural complexity and latency overhead of the RAG pipeline [36,39]. Benchmarking against a no-RAG baseline would also clarify whether the perfect groundedness score ($G = 1.0$) is attributable primarily to retrieval grounding or to prompt-level constraints.
- **Zero-day and out-of-distribution attack detection:** The current evaluation is limited to the 11 known attack classes present in the ACI IoT Network Dataset 2023. The system's ability to detect novel, unseen attack types has not been assessed. Future work should

adopt a leave-one-class-out experimental design, intentionally excluding one attack class during training and evaluating detection behavior on held-out samples at test time to characterize the framework's generalization to zero-day threats.

- **MLP model explainability:** The MLP model lacks native SHAP support in the current pipeline, reducing its explainability relative to the tree-based models. Additionally, the MLP model exhibits poor performance on the UDP Flood class (F1: 0.4828), which is compensated for by the ensemble but remains an unresolved architectural weakness. Future work should explore gradient-based attribution methods (e.g., Integrated Gradients) as alternatives to TreeSHAP for the MLP model and investigate architectural modifications or re-sampling strategies to address the UDP Flood imbalance.
- **Deployment in edge and privacy-sensitive environments:** The current setup relies on external cloud APIs (Groq API for LLM inference and Tavily for Web search), which introduces data privacy risks and network dependency unsuitable for air-gapped or sensitive operational environments. A production-grade deployment should evaluate quantized, fine-tuned LLM models running locally on edge IoT devices to achieve lower latency and full data sovereignty.

Taken together, these limitations delineate a clear path for future research: formalization of ablation studies for both the SHAP and RAG components to quantify their individual contributions, improvement of system latency through model compression and caching, extension of the evaluation to out-of-distribution attack scenarios, and adaptation of the architecture for privacy-preserving edge deployment. Addressing these directions would further validate the framework's suitability for real-world, production-grade network security operations.

5. Conclusions

This paper presents a graph-based multi-agent framework for network intrusion detection that unifies efficient pre-detection, ensemble classification, and LLM-powered incident-report generation into a single, interpretable pipeline. The system employs a three-phase architecture: a lightweight Random Forest binary classifier with top-10 MDI-selected features for edge-deployable pre-detection; an ensemble of MLP, Random Forest, and XGBoost with soft voting and SHAP-based feature attribution for explainable multi-class attack identification; and a ReAct-based summary agent that synthesizes classification results with externally retrieved threat intelligence via RAG to generate structured, evidence-grounded incident reports.

Experimental evaluation on the ACI IoT Network Dataset 2023 demonstrates that the framework achieves detection accuracy exceeding 99.8% across all 11 traffic classes; perfect groundedness ($G = 1.0$), confirming that all generated report claims derive from the retrieved context; and moderate faithfulness ($F = 0.64$), reflecting appropriate analytical synthesis. The ensemble soft-voting mechanism provides measurable resilience to individual model weaknesses, and the custom LLM-as-Judge evaluation metrics provide a rigorous, scalable alternative to human annotation for assessment of non-deterministic LLM output quality.

Collectively, these results demonstrate that integrating explainability mechanisms like SHAP for feature attribution and RAG for evidence grounding with high-accuracy detection models can produce automated incident-response systems that are not only accurate but also transparent and auditable. This work provides a foundation for future research into production-ready, privacy-preserving deployments of agentic IDSs in resource-constrained edge environments.

Author Contributions: Conceptualization, C.-H.C. and Y.P.; methodology, C.-H.C., A.S. and Y.P.; software, C.-H.C. and A.S.; validation, C.-H.C.; formal analysis, C.-H.C.; writing—original draft preparation, C.-H.C. and A.S.; writing—review and editing, C.-H.C., A.S. and Y.P.; visualization, C.-H.C.; supervision, Y.P.; project administration, Y.P.; funding acquisition, Y.P. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by NSF REU Award #2244597.

Data Availability Statement: The ACI IoT Network Traffic Dataset 2023 used in this study is publicly available on IEEE DataPort at <https://ieee-dataport.org/documents/aci-iot-network-traffic-dataset-2023> (accessed on 13 March 2026, DOI: 10.21227/qacj-3x32).

Acknowledgments: During the preparation of this manuscript, the authors used Claude Sonnet 4.6 and Claude Opus 4.6 (Anthropic) for manuscript organization, improvement of readability, and refinement of written content; Grammarly (version 1.159.0.0) for grammar and language editing; and Writefull (cloud-integrated via Overleaf AI Assist) for LaTeX language feedback and editing. The authors have reviewed and edited the suggestions and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Appendix A. Model Architectures and Configurations

The detailed architectures and hyperparameter configurations for the binary classifier and the three ensemble components are presented in Tables A1–A4.

Table A1. Lightweight Binary Random Forest Configuration.

Component	Configuration	Key Parameters
Feature Selection	Top 10 by importance	From full binary RF importance ranking
Base Learners	50 trees (default)	Optuna: 20–100 (step 10)
Max Depth	15 (default)	Optuna: 10, 15, 20, or None
Min Samples Split	10 (default)	Optuna: 5–20
Min Samples Leaf	5 (default)	Optuna: 2–10
Max Features	$\sqrt{n}$ (default)	Optuna: $\sqrt{n}$ or $\log_2(n)$
Min Weight Fraction Leaf	0.0 (default)	Optuna: 0.0–0.05 (step 0.01)
Bootstrap	True (default)	Optuna: True/False
Class Weight	Balanced (default)	Optuna: Balanced/Balanced-subsample
Hyperparameter Tuning	Optuna	20 trials, 5-fold CV

Table A2. MLP (Multi-Layer Perceptron) architecture.

Component	Configuration	Key Parameters
Input Layer	71 features	—
Hidden Layer 1	256 units	BatchNorm, ReLU, Dropout (0.3)
Hidden Layer 2	256 units	BatchNorm, ReLU, Dropout (0.3)
Hidden Layer 3	128 units	BatchNorm, ReLU, Dropout (0.2)
Hidden Layer 4	64 units	BatchNorm, ReLU, Dropout (0.2)
Output Layer	n classes	Softmax
Training	AdamW optimizer	LR: 1×10^{-3} , Weight decay: 1×10^{-4}
	CosineAnnealingWarmRestarts	$T_0 = 10$, $T_{mult} = 2$, $\eta_{min} = 1 \times 10^{-6}$
	Batch size	512
	Cross-validation	10-fold stratified

Table A3. Random Forest configuration.

Component	Configuration	Key Parameters
Base Learners	100 trees (default)	Optuna: 50–150
Max Depth	30 (default)	Optuna: 20, 30, or None
Min Samples Split	5 (default)	Optuna: 2–10
Min Samples Leaf	2 (default)	Optuna: 1–4
Max Features	$\sqrt{n}$ (default)	Optuna: $\sqrt{n}$ or $\log_2(n)$
Bootstrap	True (default)	Optuna: True/False
Class Weight	Balanced (default)	Optuna: Balanced/Balanced-subsample
Hyperparameter Tuning	Optuna	30 trials, 10-fold CV

Table A4. XGBoost Configuration.

Component	Configuration	Key Parameters
Objective	multi:softprob	Multiclass classification
Evaluation Metric	mlogloss	Multi-class log loss
Tree Method	hist	Histogram-based
Device	CUDA (if available)	GPU acceleration
Boosting Rounds	100	Early stopping: 10 rounds
Training	10-fold CV	Stratified, random state: 42

Appendix B. Example Full Execution Traces

The following tables represent the full execution traces for the benign and malicious runs, capturing the decision-making process of the Graph-Based LLM Agent Workflow.

Appendix B.1. Benign Run Trace (Normal Traffic)

This trace shows the efficiency of the “Early Exit” mechanism. The system correctly identifies the traffic as safe in the first step and bypasses the expensive RAG and LLM analysis stages, as shown in Table A5.

Table A5. Full execution trace of benign traffic (Run ID: b9fe6c5f). Bold denotes pipeline stages and field names; italics denote system actions and skipped steps.

Pipeline Stage	Details & Output
Step 1: Input Processing	
Network Flow Data	Protocol: 17,504, Flow Duration: 4.50 s, Total Fwd Packets: 0.69 (Normalized), SYN Flag: −0.88
Step 2: Ensemble Classification	
Binary Check	Result: Benign (Safe)
Confidence	Benign Confidence: 100.00% (Malicious Probability: 0.00%)
Action	Traffic Classified as Safe. Terminating analysis pipeline early.
Step 3: Execution Summary	
Final Status	Benign. No further analysis required.
Resource Usage	Skipped: Threat Intelligence Retrieval, SHAP Calculation, LLM Report Generation.

Appendix B.2. Malicious Run Trace (Slowloris Attack)

This trace demonstrates the full pipeline activation—initial classification, threat intelligence retrieval, SHAP explanation, and final LLM report generation—as shown in Table A6.

Table A6. Full execution trace of malicious traffic (Run ID: 753f26bb). Bold denotes pipeline stages and field names; italics denote system actions and retrieved evidence.

Pipeline Stage	Details & Output
Step 1: Input Processing	
Network Flow Data	Protocol: 65,184, Flow Duration: −0.447 (Normalized), SYN Flag: 1.94, RST Flag: 2.84
Step 2: Ensemble Classification	
Binary Check	Result: Malicious (Confidence: 100%)
Multi-Class Prediction	MLP: Slowloris (99.99%), Random Forest: Slowloris (100.00%), XGBoost: Slowloris (99.99%)
Consensus	Unanimous Agreement: Slowloris Attack
Step 3: Threat Analysis & Explainability	
SHAP Feature Analysis	Top Contributors: Protocol, Flow Duration, Total Fwd Packets, Fwd Packet Length Max.
Threat Intelligence (RAG)	<i>Query: “Slowloris attack”. Retrieval: “Slowloris is an application layer DDoS attack which uses partial HTTP requests to open connections between a single computer and a targeted Web server.” (Source: NetScout, Wikipedia)</i>
Step 4: Final Report Generation (LLM)	
Security Analysis	Classification: Malicious (High Confidence). Attack Type: Slowloris. Reasoning: The ensemble models unanimously identified the traffic as Slowloris. External intelligence confirms the attack pattern matches the feature profile (partial HTTP requests, connection holding). Mitigation: Implement rate limiting, configure firewalls to drop partial connections, and use timeout configurations.

References

- Singh, N.; Buyya, R.; Kim, H. Securing Cloud-Based Internet of Things: Challenges and Mitigations. *arXiv* **2024**, arXiv:2402.00356. [[CrossRef](#)] [[PubMed](#)]
- Ahmadi, S. Next Generation AI-Based Firewalls: A Comparative Study. *Int. J. Comput.* **2023**, *49*, 245–262.
- Tariq, U.; Ahmed, I.U.; Bashir, A.; Shaikat, K. A Critical Cybersecurity Analysis and Future Research Directions for the Internet of Things: A Comprehensive Review. *Sensors* **2023**, *23*, 4117. [[CrossRef](#)] [[PubMed](#)]
- Kheddar, H. Transformers and Large Language Models for Efficient Intrusion Detection Systems: A Comprehensive Survey. *Inf. Fusion* **2024**, *124*, 103347. [[CrossRef](#)]
- Jüttner, V.; Grimmer, M.; Buchmann, E. ChatIDS: Explainable Cybersecurity Using Generative AI. *arXiv* **2023**, arXiv:2306.14504. [[CrossRef](#)]
- Ali, T.; Kostakos, P. HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs). *arXiv* **2023**, arXiv:2309.16021. [[CrossRef](#)]
- Lundberg, S.M.; Lee, S.I. A Unified Approach to Interpreting Model Predictions. In Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA, 4–9 December 2017; pp. 4765–4774.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. React: Synergizing reasoning and acting in language models. In Proceedings of the International Conference on Learning Representations (ICLR), Kigali, Rwanda, 1–5 May 2023.
- Verma, A.; Ranga, V. Machine Learning Based Intrusion Detection Systems for IoT Applications. *Wirel. Pers. Commun.* **2019**, *111*, 2287–2310. [[CrossRef](#)]
- Yin, C.; Zhu, Y.; Fei, J.L.; He, X.Z. A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks. *IEEE Access* **2017**, *5*, 21954–21961. [[CrossRef](#)]
- Shone, N.; Ngoc, T.N.; Phai, V.D.; Shi, Q. A Deep Learning Approach to Network Intrusion Detection. *IEEE Trans. Emerg. Top. Comput. Intell.* **2018**, *2*, 41–50. [[CrossRef](#)]
- Vinayakumar, R.; Alazab, M.; Member, I.K.P.S.S.; Poornachandran, P.; Al-Nemrat, A.; Venkatraman, S. Deep Learning Approach for Intelligent Intrusion Detection System. *IEEE Access* **2019**, *7*, 41525–41550. [[CrossRef](#)]
- Abdulganiyu, O.H.; Ait Tchakoucht, T.; Saheed, Y.K. A Systematic Literature Review for Network Intrusion Detection System (IDS). *Int. J. Inf. Secur.* **2023**, *22*, 1125–1162. [[CrossRef](#)]
- Diana, L.; Dini, P.; Paolini, D. Overview on Intrusion Detection Systems for Computers Networking Security. *Computers* **2025**, *14*, 87. [[CrossRef](#)]
- Moustafa, N.; Turnbull, B.; Choo, K.K.R. An Ensemble Intrusion Detection Technique Based on Proposed Statistical Flow Features for Protecting Network Traffic of Internet of Things. *IEEE Internet Things J.* **2019**, *6*, 4815–4830. [[CrossRef](#)]
- Hossain, M.A.; Islam, M.S. Ensuring network security with a robust intrusion detection system using ensemble-based machine learning. *Array* **2023**, *19*, 100306. [[CrossRef](#)]

17. Gao, X.; Shan, C.; Hu, C.; Niu, Z.; Liu, Z. An Adaptive Ensemble Machine Learning Model for Intrusion Detection. *IEEE Access* **2019**, *7*, 82512–82521. [\[CrossRef\]](#)
18. Alotaibi, Y.; Ilyas, M. Ensemble-Learning Framework for Intrusion Detection to Enhance Internet of Things' Devices Security. *Sensors* **2023**, *23*, 5568. [\[CrossRef\]](#) [\[PubMed\]](#)
19. Almotairi, A.; Atawneh, S.; Khashan, O.A.; Khafajah, N.M. Enhancing Intrusion Detection in IoT Networks Using Machine Learning-Based Feature Selection and Ensemble Models. *Syst. Sci. Control Eng.* **2024**, *12*, 2321381. [\[CrossRef\]](#)
20. Ahmed, U.; Zheng, J.; Almogren, A.; Sadiq, A.; Sadiq, M.; Choi, J. Hybrid Bagging and Boosting with SHAP Based Feature Selection for Enhanced Predictive Modeling in Intrusion Detection Systems. *Sci. Rep.* **2024**, *14*, 29591. [\[CrossRef\]](#)
21. Gaspar, D.; Silva, P.; Silva, C. Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron. *IEEE Access* **2024**, *12*, 30164–30175. [\[CrossRef\]](#)
22. Abou El Houda, Z.; Brik, B.; Khoukhi, L. “Why Should I Trust Your IDS?”: An Explainable Deep Learning Framework for Intrusion Detection Systems in Internet of Things Networks. *IEEE Open J. Commun. Soc.* **2022**, *3*, 1164–1176. [\[CrossRef\]](#)
23. Wang, M.; Zheng, K.; Yang, Y.; Wang, X. An Explainable Machine Learning Framework for Intrusion Detection Systems. *IEEE Access* **2020**, *8*, 73127–73141. [\[CrossRef\]](#)
24. Younis, R.; Ahmad, A.; Abu Al-Haija, Q. Explaining Intrusion Detection-Based Convolutional Neural Networks Using Shapley Additive Explanations (SHAP). *Big Data Cogn. Comput.* **2022**, *6*, 126. [\[CrossRef\]](#)
25. Naveed, H.; Khan, A.U.; Qiu, S.; Saqib, M.; Anwar, S.; Usman, M.; Barnes, N.; Mian, A. A Comprehensive Overview of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2023**, *16*, 1–72. [\[CrossRef\]](#)
26. Bilal, A.; Ebert, D.; Lin, B. LLMs for Explainable AI: A Comprehensive Survey. *arXiv* **2025**, arXiv:2504.00125. [\[CrossRef\]](#)
27. Gupta, M.; Akiri, C.; Aryal, K.; Parker, E.; Praharaj, L. From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy. *IEEE Access* **2023**, *11*, 80218–80245. [\[CrossRef\]](#)
28. Xu, S.; Yan, Z.; Dai, C.; Wu, F. MEGA-RAG: A retrieval-augmented generation framework with multi-evidence guided answer refinement for mitigating hallucinations of LLMs in public health. *Front. Public Health* **2025**, *13*, 1635381. [\[CrossRef\]](#)
29. Xiong, G.; Jin, Q.; Lu, Z.; Zhang, A. Benchmarking Retrieval-Augmented Generation for Medicine. *arXiv* **2024**, arXiv:2402.13178. [\[CrossRef\]](#)
30. Alhuzali, A. LLM-powered threat intelligence: A retrieval-augmented generation approach for cyber attack investigation. *PeerJ Comput. Sci.* **2025**, *11*, e3371. [\[CrossRef\]](#)
31. Borah, A.; Alam, M.T.; Rastogi, N. Adapting Large Language Models to Emerging Cybersecurity Using Retrieval Augmented Generation. *arXiv* **2025**, arXiv:2510.27080. [\[CrossRef\]](#)
32. Tellache, A.; Amara Korba, A.; Mokhtari, A.; Moldovan, H.; Ghamri-Doudane, Y. Advancing Autonomous Incident Response: Leveraging LLMs and Cyber Threat Intelligence. *arXiv* **2025**, arXiv:2508.10677. [\[CrossRef\]](#)
33. Chen, J.; Lin, H.; Han, X.; Sun, L. Benchmarking Large Language Models in Retrieval-Augmented Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence, Vancouver, BC, Canada, 20–27 February 2024*; Association for the Advancement of Artificial Intelligence: Washington, DC, USA 2024; Volume 38, pp. 17754–17762. [\[CrossRef\]](#)
34. Gan, A.; Yu, H.; Zhang, K.; Yan, W.; Huang, Z.; Tong, S.; Chen, E.; Hu, G. Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey. *arXiv* **2025**, arXiv:2504.14891. [\[CrossRef\]](#)
35. Gu, J.; Jiang, X.; Shi, Z.; Tan, H.; Zhai, X.; Xu, C.; Li, W.; Shen, Y.; Ma, S.; Liu, H.; et al. A Survey on LLM-as-a-Judge. *arXiv* **2024**, arXiv:2411.15594. [\[CrossRef\]](#)
36. Shahul, E.; James, J.; Anke, L.E.; Schockaert, S. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, St. Julians, Malta, 17–22 March 2024*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024; pp. 150–158. [\[CrossRef\]](#)
37. Nack, E.A.; McKenzie, M.C.; Bastian, N.D. ACI-IoT-2023: A Robust Dataset for Internet of Things Network Security Analysis. In *Proceedings of the MILCOM 2024—2024 IEEE Military Communications Conference (MILCOM), Washington, DC, USA, 28 October–1 November 2024*; IEEE: New York, NY, USA, 2024; pp. 1–6. [\[CrossRef\]](#)
38. Assiri, A.; Nazir, S.; Velastin, S. Breast Tumor Classification Using an Ensemble Machine Learning Method. *J. Imaging* **2020**, *6*, 39. [\[CrossRef\]](#)
39. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Kuttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv* **2020**, arXiv:2005.11401.
40. Asal, B.; Cakin, A.; Dilek, S. Enhancing Industrial IoT Cybersecurity with Explainable AI: A SHAP and LIME-Based Intrusion Detection Methodology. In *Proceedings of the 2025 7th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA), Ankara, Türkiye, 23–24 May 2025*; IEEE: New York, NY, USA, 2025; pp. 1–8. [\[CrossRef\]](#)
41. Yu, H.; Gan, A.; Zhang, K.; Tong, S.; Liu, Q.; Liu, Z. Evaluation of Retrieval-Augmented Generation: A Survey. *arXiv* **2024**, arXiv:2405.07437. [\[CrossRef\]](#)

42. Saad-Falcon, J.; Khattab, O.; Potts, C.; Zaharia, M. ARES: An Automated RAG Evaluation System. *arXiv* **2023**, arXiv:2311.09476.
43. Zhao, P.; Zhang, H.; Yu, Q.; Wang, Z.; Geng, Y.; Fu, F.; Yang, L.; Zhang, W.; Cui, B. Retrieval-Augmented Generation for AI-Generated Content: A Survey. *arXiv* **2024**, arXiv:2402.19473. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.